

# COMPUTATIONAL MODEL FOR PARSING EXPRESSION GRAMMARS

ALEXANDER RUBTSOV

HSE UNIVERSITY, MOSCOW, RUSSIA

BASED ON MFCS 2024 PAPER  
(JOINT WORK WITH NIKITA CHUDINOV)

NOVEMBER 5, 2025

# **PARSING EXPRESSION GRAMMARS**

# **PARSING EXPRESSION GRAMMARS**

**MOTIVATION, DEFINITION, EXAMPLES**

# PRACTICAL MOTIVATION

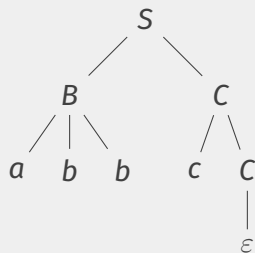
## Classical Parsing Approaches

- Top-Down:  $LL(k)$   
*easy to design, but the power is limited*
- Bottom-Up:  $LR(k)$   
*powerful (generate all DCFLs), but hard to design*

## Parsing Expression Grammars

- PEGs can be considered as a generalization of LL-grammars
- PEGs are more powerful than LR-grammars, but there is no (currently?) direct translation LR-grammars to PEGs
- PEGs now being used in compilers  
*Python replaced  $LL(1)$ -parser by PEG*
- PEGs are popular for solving parsing problems

# PEGS AS LL( $k$ )-GENERALIZATION

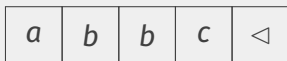


**LL(1)-Grammar:**

$$S \rightarrow BC$$

$$B \rightarrow abb \mid b$$

$$C \rightarrow cC \mid \varepsilon$$



# PEGS AS $LL(k)$ -GENERALIZATION

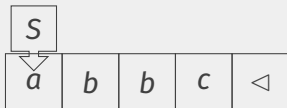
$S$

**LL(1)-Grammar:**

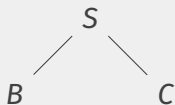
$S \rightarrow BC$

$B \rightarrow abb \mid b$

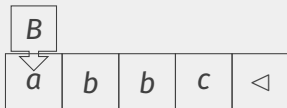
$C \rightarrow cC \mid \varepsilon$



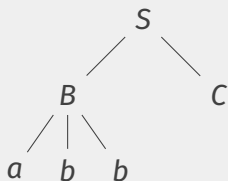
# PEGS AS LL( $k$ )-GENERALIZATION



**LL(1)-Grammar:**

$$S \rightarrow BC$$
$$B \rightarrow abb \mid b$$
$$C \rightarrow cC \mid \varepsilon$$


# PEGS AS LL( $k$ )-GENERALIZATION

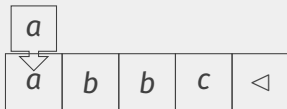


**LL(1)-Grammar:**

$$S \rightarrow BC$$

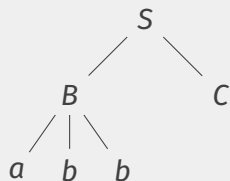
$$B \rightarrow abb \mid b$$

$$C \rightarrow cC \mid \varepsilon$$





# PEGS AS LL( $k$ )-GENERALIZATION

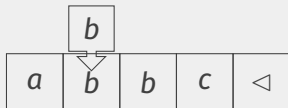


**LL(1)-Grammar:**

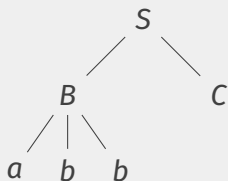
$$S \rightarrow BC$$

$$B \rightarrow abb \mid b$$

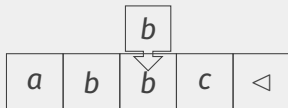
$$C \rightarrow cC \mid \varepsilon$$



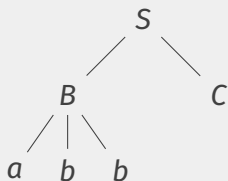
# PEGS AS LL( $k$ )-GENERALIZATION



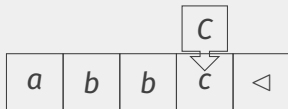
**LL(1)-Grammar:**

$$S \rightarrow BC$$
$$B \rightarrow abb \mid b$$
$$C \rightarrow cC \mid \varepsilon$$


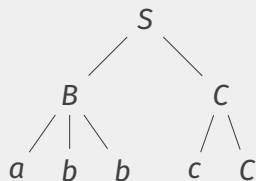
# PEGS AS LL( $k$ )-GENERALIZATION



**LL(1)-Grammar:**

$$S \rightarrow BC$$
$$B \rightarrow abb \mid b$$
$$C \rightarrow cC \mid \varepsilon$$


# PEGS AS LL( $k$ )-GENERALIZATION

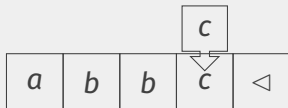


**LL(1)-Grammar:**

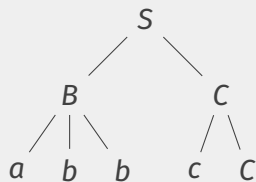
$$S \rightarrow BC$$

$$B \rightarrow abb \mid b$$

$$C \rightarrow cC \mid \varepsilon$$



# PEGS AS LL( $k$ )-GENERALIZATION

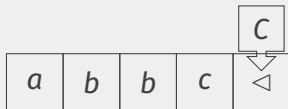


**LL(1)-Grammar:**

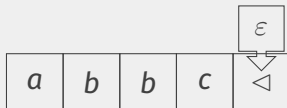
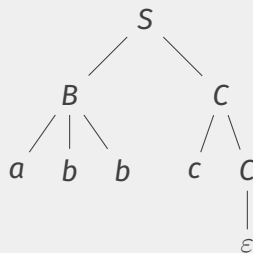
$$S \rightarrow BC$$

$$B \rightarrow abb \mid b$$

$$C \rightarrow cC \mid \varepsilon$$



# PEGS AS LL( $k$ )-GENERALIZATION



**LL(1)-Grammar:**

$S \rightarrow BC$

$B \rightarrow abb \mid b$

$C \rightarrow cC \mid \varepsilon$

# PEGs AS LL( $k$ )-GENERALIZATION

## CFG

$S \rightarrow AB \mid BC$

$A \rightarrow aA \mid a$

$B \rightarrow abb \mid b$

$C \rightarrow cC \mid \varepsilon$

## PEG

$S \leftarrow AB / BC$

$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$

# PEGS AS LL( $k$ )-GENERALIZATION

$S$

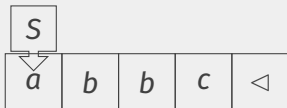
**PEG:**

$S \leftarrow AB / BC$

$A \leftarrow aA / a$

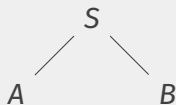
$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$





# PEGs AS LL( $k$ )-GENERALIZATION



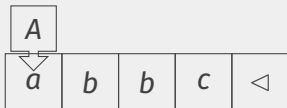
**PEG:**

$S \leftarrow AB / BC$

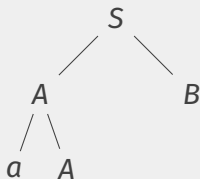
$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$



# PEGS AS LL( $k$ )-GENERALIZATION



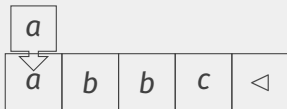
**PEG:**

$S \leftarrow AB / BC$

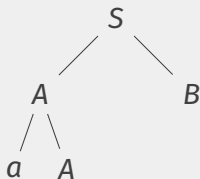
$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$



# PEGS AS LL( $k$ )-GENERALIZATION



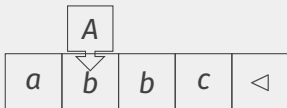
**PEG:**

$S \leftarrow AB / BC$

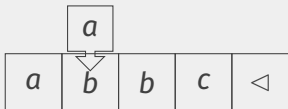
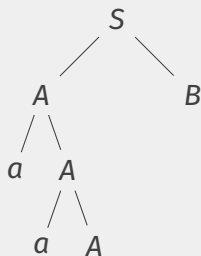
$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$



# PEGS AS LL( $k$ )-GENERALIZATION



**PEG:**

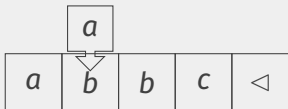
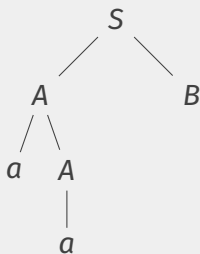
$S \leftarrow AB / BC$

$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$

# PEGS AS LL( $k$ )-GENERALIZATION



**PEG:**

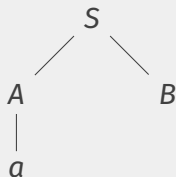
$S \leftarrow AB / BC$

$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \epsilon$

# PEGS AS LL( $k$ )-GENERALIZATION



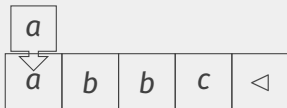
**PEG:**

$S \leftarrow AB / BC$

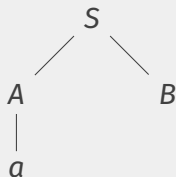
$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$



# PEGS AS LL( $k$ )-GENERALIZATION



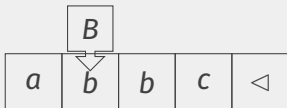
**PEG:**

$S \leftarrow AB / BC$

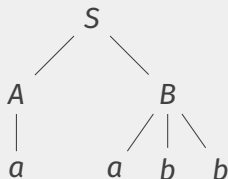
$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$



# PEGS AS LL( $k$ )-GENERALIZATION



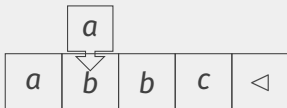
**PEG:**

$S \leftarrow AB / BC$

$A \leftarrow aA / a$

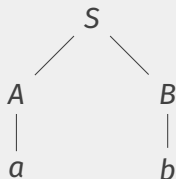
$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$





# PEGS AS LL( $k$ )-GENERALIZATION



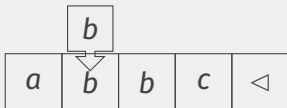
**PEG:**

$S \leftarrow AB / BC$

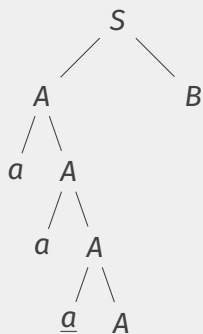
$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$



# PEGS AS LL( $k$ )-GENERALIZATION



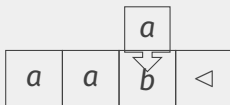
**PEG:**

$S \leftarrow AB / BC$

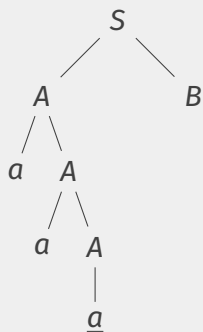
$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$



# PEGS AS LL( $k$ )-GENERALIZATION



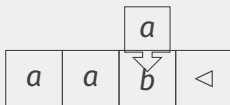
**PEG:**

$S \leftarrow AB / BC$

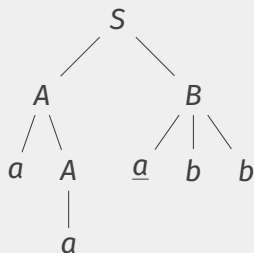
$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$



# PEGS AS LL( $k$ )-GENERALIZATION



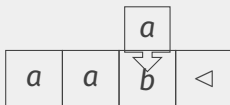
**PEG:**

$S \leftarrow AB / BC$

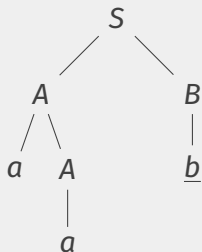
$A \leftarrow aA / a$

$B \leftarrow abb / b$

$C \leftarrow cC / \varepsilon$



# PEGS AS LL( $k$ )-GENERALIZATION



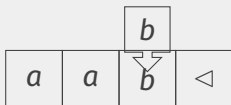
**PEG:**

$S \leftarrow AB / BC$

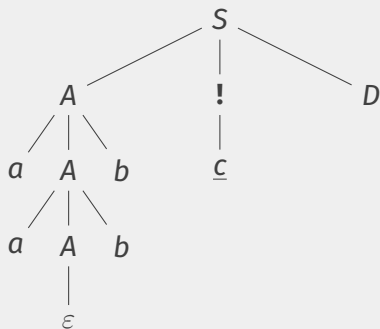
$A \leftarrow aA / a$

$B \leftarrow abb / \underline{b}$

$C \leftarrow cC / \varepsilon$



# PEGs EXAMPLES: OPERATOR !



**PEG:**

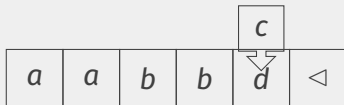
$$S \leftarrow A(!c)D / A'B$$

$$A \leftarrow aAb / \epsilon$$

$$A' \leftarrow aA' / \epsilon$$

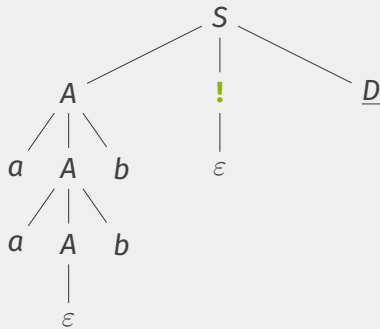
$$B \leftarrow bBc / \epsilon$$

$$D \leftarrow dD / \epsilon$$



$$a^n b^n d^* \cup a^* b^n c^n$$

# PEGs EXAMPLES: OPERATOR !



**PEG:**

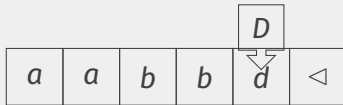
$$S \leftarrow A(!c)D / A'B$$

$$A \leftarrow aAb / \epsilon$$

$$A' \leftarrow aA' / \epsilon$$

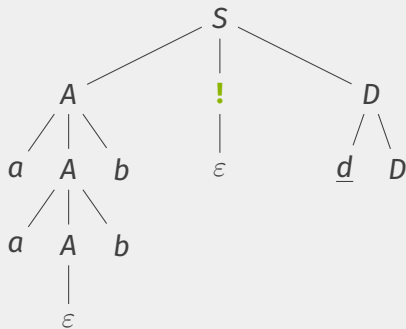
$$B \leftarrow bBc / \epsilon$$

$$D \leftarrow dD / \epsilon$$



$$a^n b^n d^* \cup a^* b^n c^n$$

# PEGs EXAMPLES: OPERATOR !



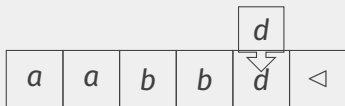
**PEG:**

$$S \leftarrow A(!c)D / A'B$$

$$A \leftarrow aAb / \varepsilon$$

$$A' \leftarrow aA' / \varepsilon$$

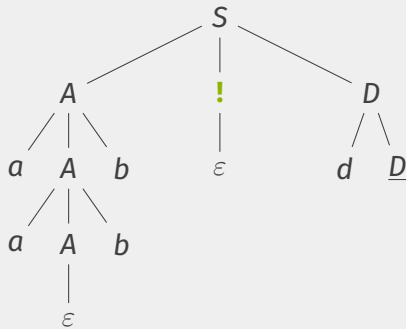
$$B \leftarrow bBc / \varepsilon$$

$$D \leftarrow dD / \varepsilon$$


$$a^n b^n d^* \cup a^* b^n c^n$$



# PEGs EXAMPLES: OPERATOR !



**PEG:**

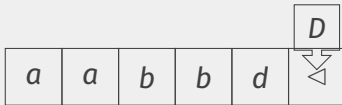
$$S \leftarrow A(!c)D / A'B$$

$$A \leftarrow aAb / \varepsilon$$

$$A' \leftarrow aA' / \varepsilon$$

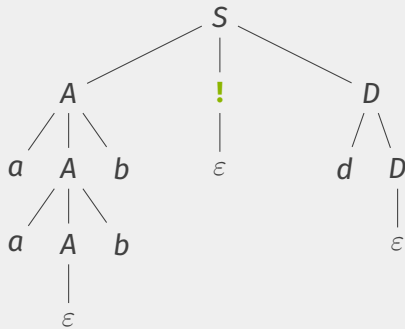
$$B \leftarrow bBc / \varepsilon$$

$$D \leftarrow dD / \varepsilon$$



$$a^n b^n d^* \cup a^* b^n c^n$$

# PEGs EXAMPLES: OPERATOR !



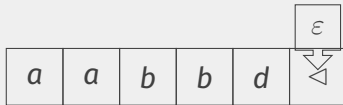
**PEG:**

$$S \leftarrow A(!c)D / A'B$$

$$A \leftarrow aAb / \epsilon$$

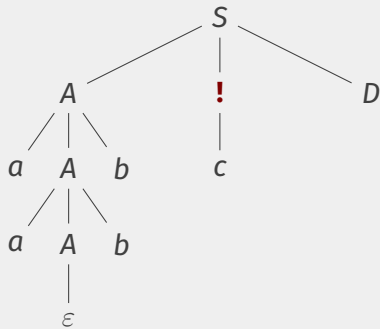
$$A' \leftarrow aA' / \epsilon$$

$$B \leftarrow bBc / \epsilon$$

$$D \leftarrow dD / \epsilon$$


$$a^n b^n d^* \cup a^* b^n c^n$$

# PEGs EXAMPLES: OPERATOR !



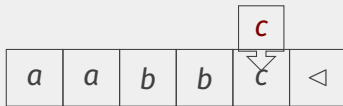
**PEG:**

$$S \leftarrow A(!c)D / A'B$$

$$A \leftarrow aAb / \varepsilon$$

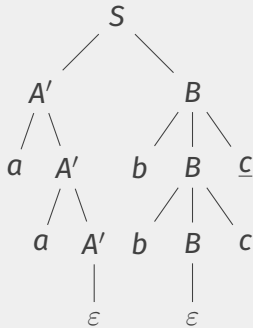
$$A' \leftarrow aA' / \varepsilon$$

$$B \leftarrow bBc / \varepsilon$$

$$D \leftarrow dD / \varepsilon$$


$$a^n b^n d^* \cup a^* b^n c^n$$

# PEGs EXAMPLES: OPERATOR !



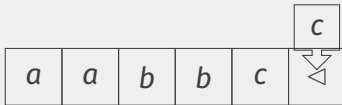
**PEG:**

$$S \leftarrow A(!c)D / A'B$$

$$A \leftarrow aAb / \varepsilon$$

$$A' \leftarrow aA' / \varepsilon$$

$$B \leftarrow bBc / \varepsilon$$

$$D \leftarrow dD / \varepsilon$$


$$a^n b^n d^* \cup a^* b^n c^n$$

# PEGs EXAMPLES: OPERATOR &

& is a syntactic sugar

$$\&X = !(X)$$

- if  $X$  yields to fail then  $!X$  yields to  $\varepsilon$
- if  $X$  does not yield to fail then  $\&X$  yields to  $\varepsilon$

$$G : S \leftarrow (\&(Ac))BC \quad A \leftarrow aAb / \varepsilon \quad B \leftarrow aB / a \quad C \leftarrow bCc / \varepsilon$$

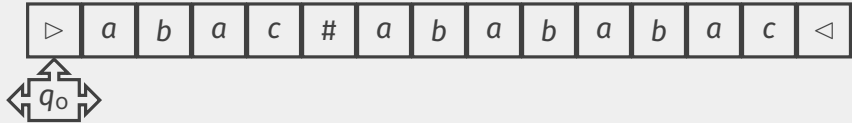
$$L(G) = \{a^n b^n c^n \mid n \geq 1\}$$

# DISCUSSION

- “Concatenation” in PEGs is not a real concatenation!
  - ▶ **Conjecture:** PELs are not closed over concatenation
- PELs are closed over Boolean operations:  $\Gamma_{\text{Bool}}(\text{PEL}) = \text{PEL}$
- $\text{PEL} \setminus \text{CFL} \neq \emptyset$
- *Open question:*  $\text{CFL} \setminus \text{PEL} \stackrel{?}{=} \emptyset$ 
  - ▶  $\Omega(n^{1+\varepsilon})$  bound on CFLs parsing implies  $\text{CFL} \setminus \text{PEL} \neq \emptyset$

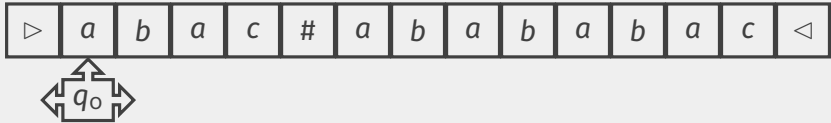
# POINTER PUSHDOWN AUTOMATA

# POINTER PUSHDOWN AUTOMATA

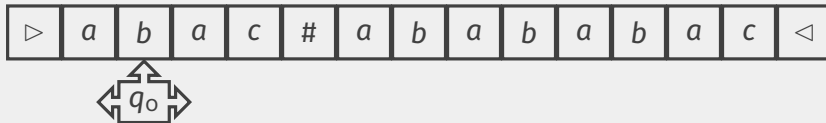




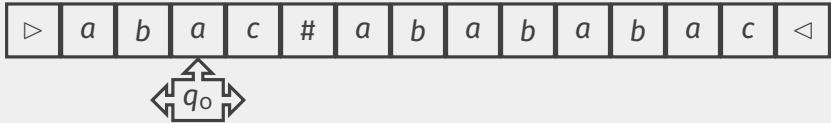
# POINTER PUSHDOWN AUTOMATA



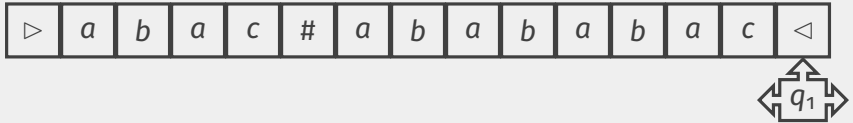
# POINTER PUSHDOWN AUTOMATA



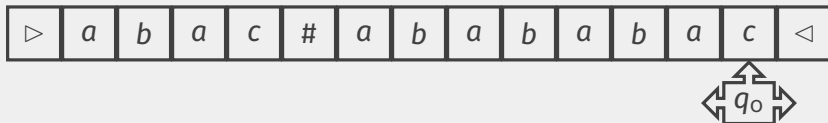
# POINTER PUSHDOWN AUTOMATA



# POINTER PUSHDOWN AUTOMATA



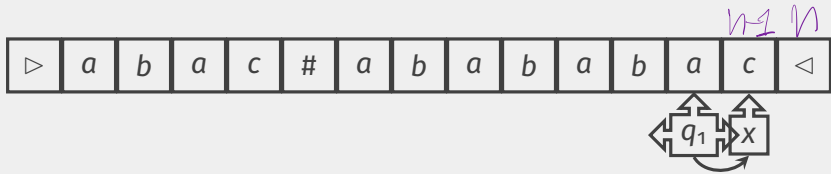
# POINTER PUSHDOWN AUTOMATA



## Stack Operations:

- $\text{push}(x) \leftarrow$

# POINTER PUSHDOWN AUTOMATA

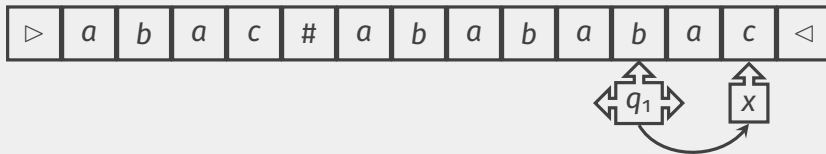


## Stack Operations:

- $\text{push}(x) \leftarrow$



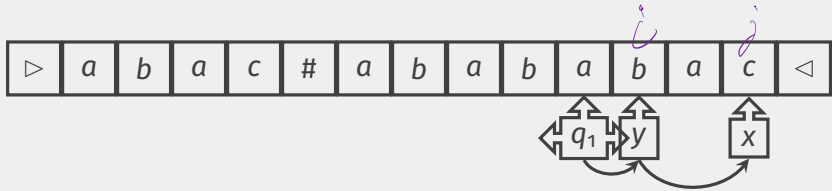
# POINTER PUSHDOWN AUTOMATA



## Stack Operations:

- $\text{push}(x) \leftarrow$
- $\text{push}(y) \leftarrow$

# POINTER PUSHDOWN AUTOMATA



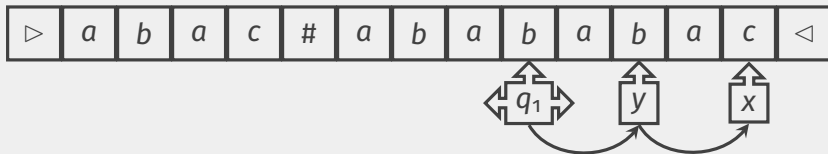
## Stack Operations:

- $\text{push}(x) \leftarrow$
- $\text{push}(y) \leftarrow$





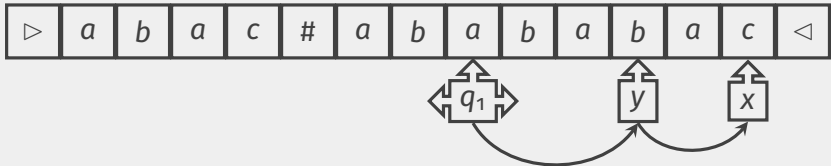
# POINTER PUSHDOWN AUTOMATA



## Stack Operations:

- $\text{push}(x) \leftarrow$
- $\text{push}(y) \leftarrow$

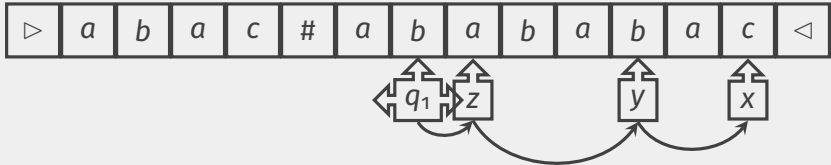
# POINTER PUSHDOWN AUTOMATA



## Stack Operations:

- $\text{push}(x) \leftarrow$
- $\text{push}(y) \leftarrow$
- $\text{push}(z) \leftarrow$

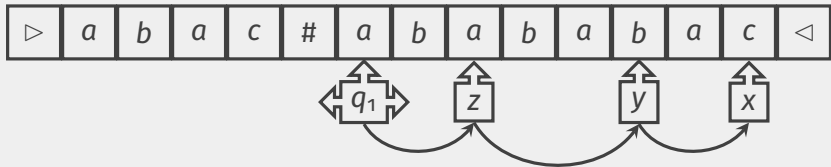
# POINTER PUSHDOWN AUTOMATA



## Stack Operations:

- $\text{push}(x) \leftarrow$
- $\text{push}(y) \leftarrow$
- $\text{push}(z) \leftarrow$

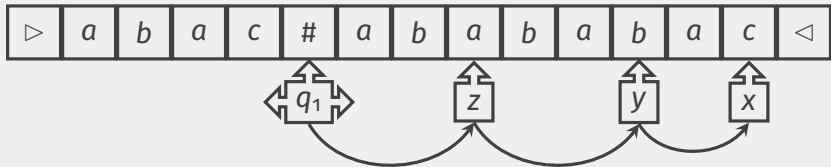
# POINTER PUSHDOWN AUTOMATA



## Stack Operations:

- $\text{push}(x) \leftarrow$
- $\text{push}(y) \leftarrow$
- $\text{push}(z) \leftarrow$

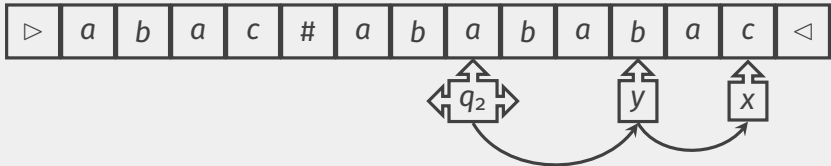
# POINTER PUSHDOWN AUTOMATA



## Stack Operations:

- $\text{push}(x) \leftarrow$
- $\text{push}(y) \leftarrow$
- $\text{push}(z) \leftarrow$
- $\text{pop}(z) \uparrow$

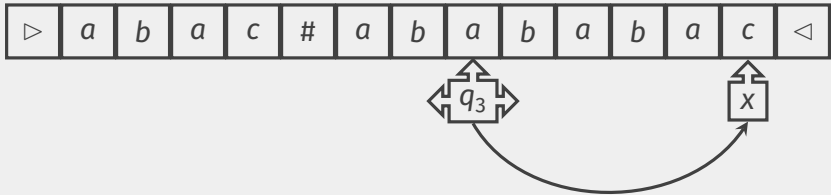
# POINTER PUSHDOWN AUTOMATA



## Stack Operations:

- $\text{push}(x) \leftarrow$
- $\text{push}(y) \leftarrow$
- $\text{push}(z) \leftarrow$
- $\text{pop}(z) \uparrow$
- $\text{pop}(y) \downarrow$

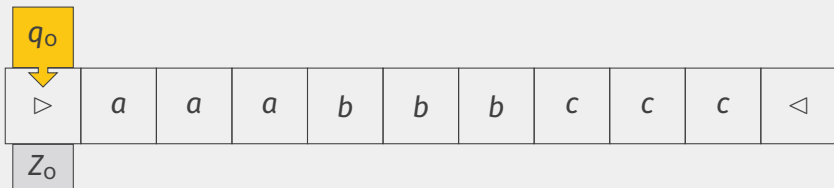
# POINTER PUSHDOWN AUTOMATA



## Stack Operations:

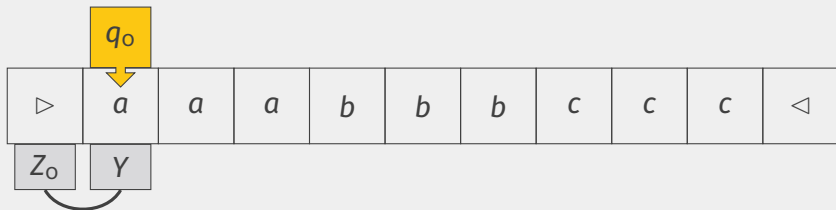
- $\text{push}(x) \leftarrow$
- $\text{push}(y) \leftarrow$
- $\text{push}(z) \leftarrow$
- $\text{pop}(z) \uparrow$
- $\text{pop}(y) \downarrow$

$$a^n b^n c^n$$

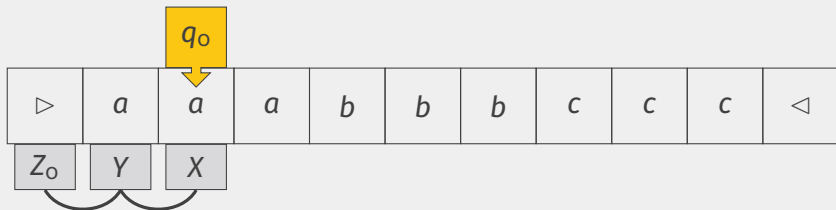




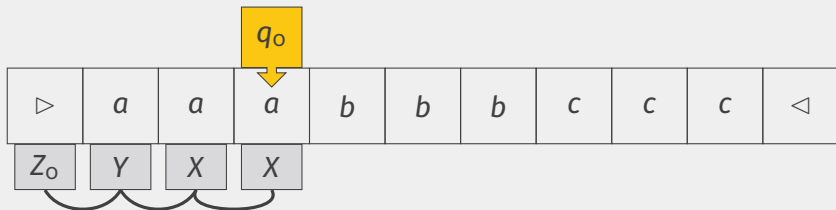
$$a^n b^n c^n$$



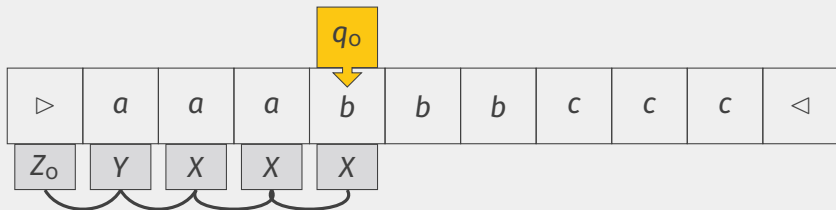
$$a^n b^n c^n$$



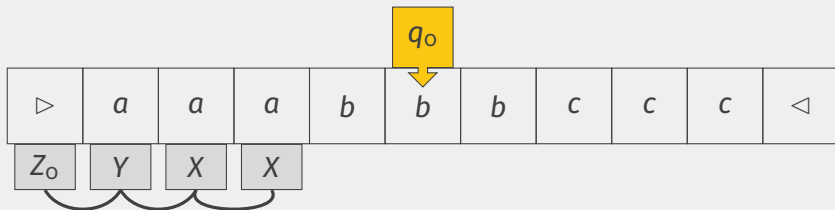
$$a^n b^n c^n$$



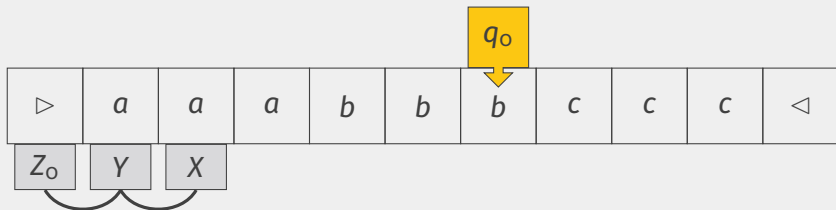
$$a^n b^n c^n$$



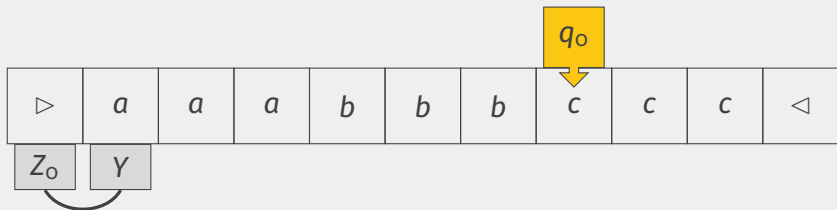
$$a^n b^n c^n$$



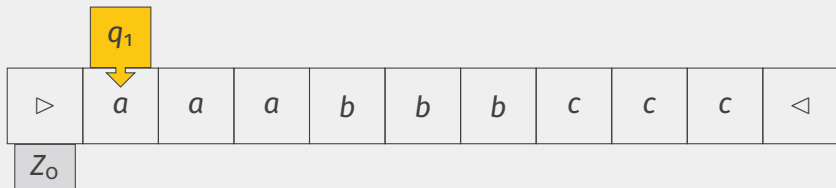
$$a^n b^n c^n$$



$$a^n b^n c^n$$

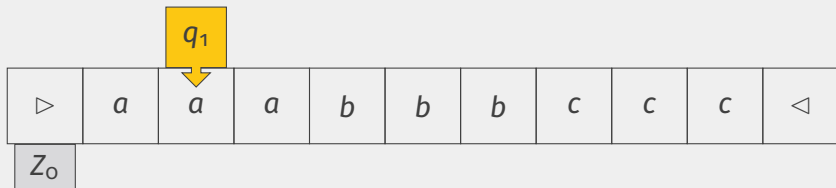


$$a^n b^n c^n$$

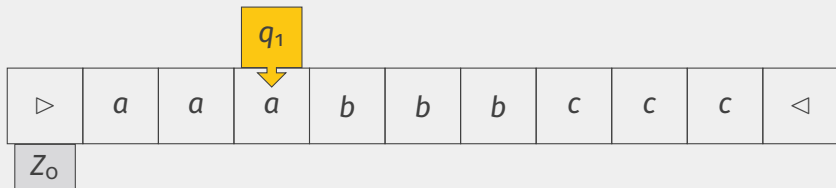




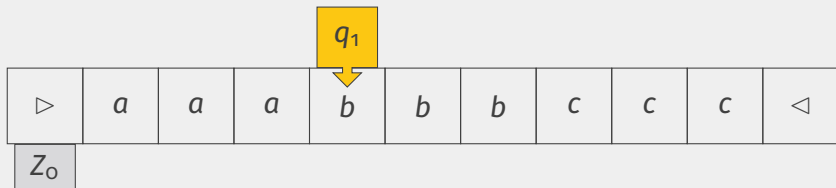
$$a^n b^n c^n$$



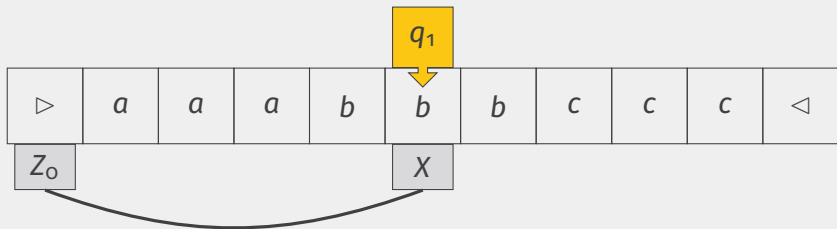
$$a^n b^n c^n$$



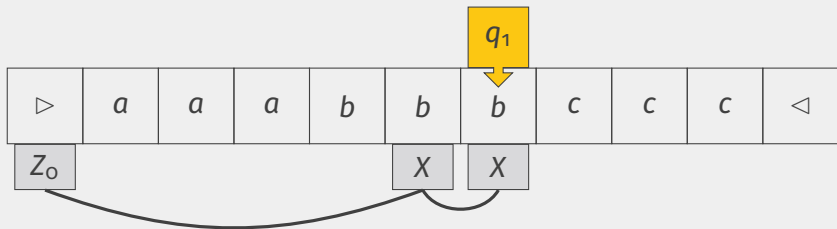
$$a^n b^n c^n$$



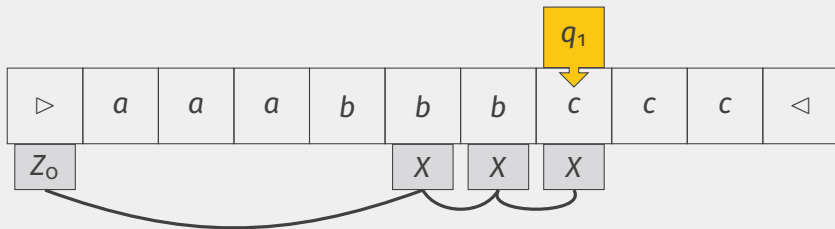
$$a^n b^n c^n$$



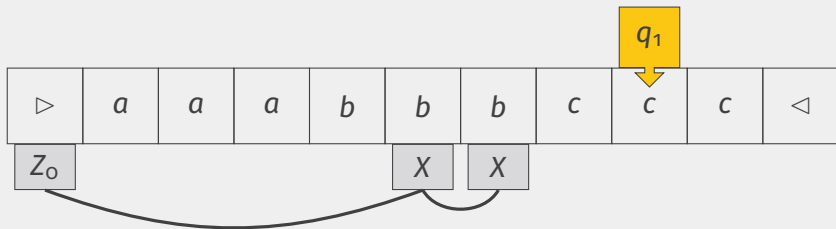
$$a^n b^n c^n$$



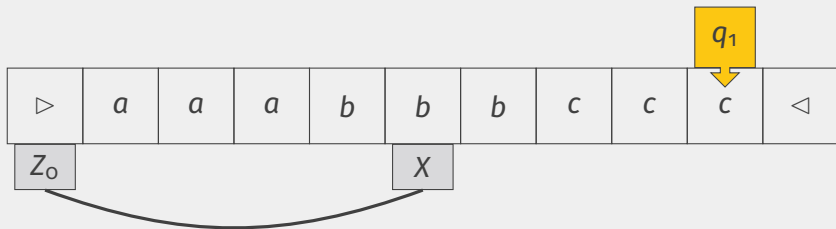
$$a^n b^n c^n$$



$$a^n b^n c^n$$

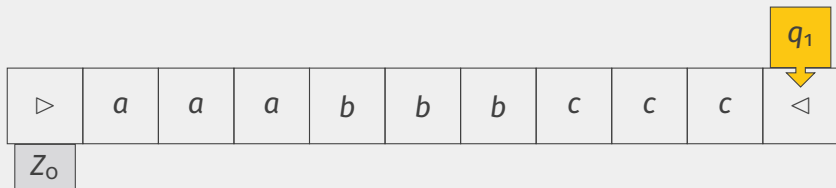


$$a^n b^n c^n$$

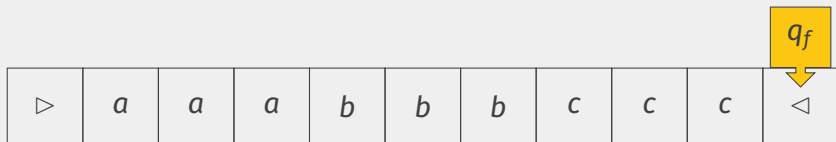




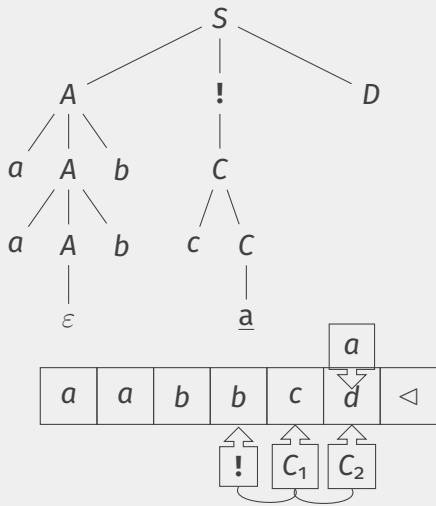
$$a^n b^n c^n$$



$$a^n b^n c^n$$



# PEG $\rightarrow$ DPPDA (IDEA)



**PEG:**

$S \leftarrow A(!C)D / A'B$

$A \leftarrow aAb / \epsilon$

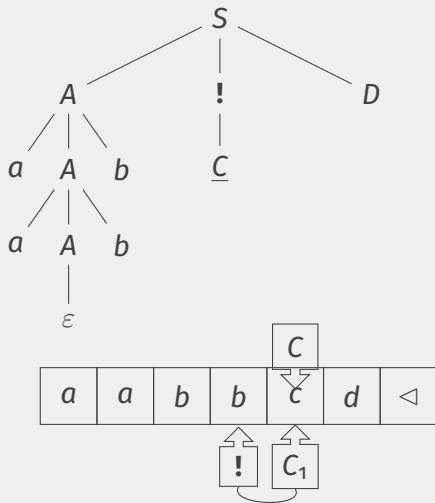
$A' \leftarrow aA' / \epsilon$

$B \leftarrow bBc / \epsilon$

$C \leftarrow cC / a$

$D \leftarrow dD / \epsilon$

# PEG $\rightarrow$ DPPDA (IDEA)



**PEG:**

$S \leftarrow A(!C)D / A'B$

$A \leftarrow aAb / \epsilon$

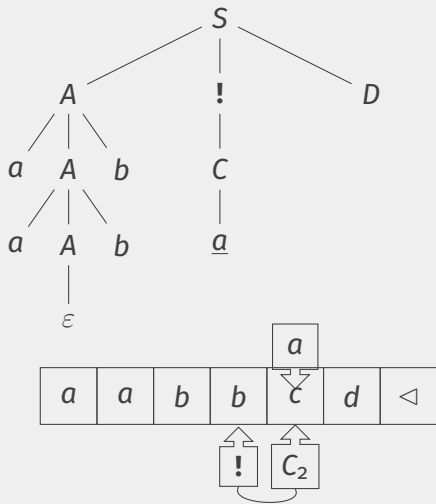
$A' \leftarrow aA' / \epsilon$

$B \leftarrow bBc / \epsilon$

$C \leftarrow cC / a$

$D \leftarrow dD / \epsilon$

# PEG $\rightarrow$ DPPDA (IDEA)



**PEG:**

$S \leftarrow A(!C)D / A'B$

$A \leftarrow aAb / \epsilon$

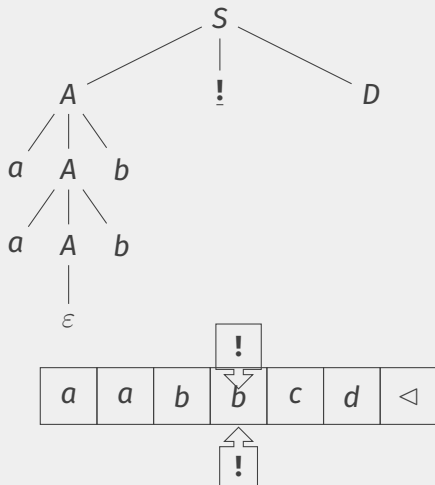
$A' \leftarrow aA' / \epsilon$

$B \leftarrow bBc / \epsilon$

$C \leftarrow cC / a$

$D \leftarrow dD / \epsilon$

# PEG $\rightarrow$ DPPDA (IDEA)



**PEG:**

$S \leftarrow A(!C)D / A'B$

$A \leftarrow aAb / \epsilon$

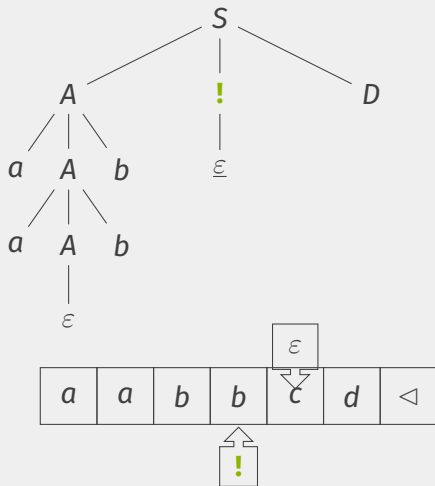
$A' \leftarrow aA' / \epsilon$

$B \leftarrow bBc / \epsilon$

$C \leftarrow cC / a$

$D \leftarrow dD / \epsilon$

# PEG $\rightarrow$ DPPDA (IDEA)



**PEG:**

$S \leftarrow A(!C)D / A'B$

$A \leftarrow aAb / \epsilon$

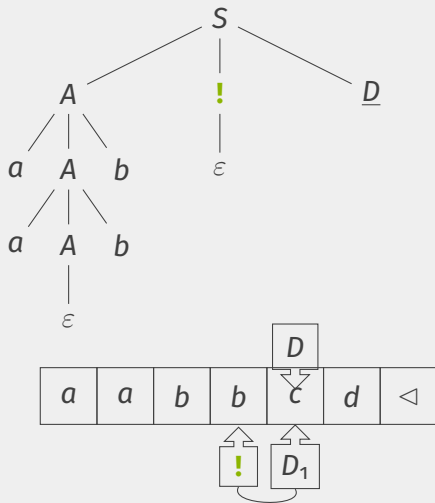
$A' \leftarrow aA' / \epsilon$

$B \leftarrow bBc / \epsilon$

$C \leftarrow cC / a$

$D \leftarrow dD / \epsilon$

# PEG $\rightarrow$ DPPDA (IDEA)



**PEG:**

$S \leftarrow A(!C)D / A'B$

$A \leftarrow aAb / \epsilon$

$A' \leftarrow aA' / \epsilon$

$B \leftarrow bBc / \epsilon$

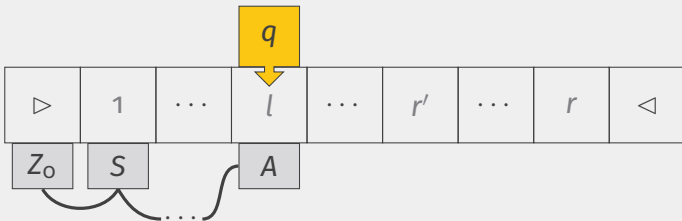
$C \leftarrow cC / a$

$D \leftarrow dD / \epsilon$



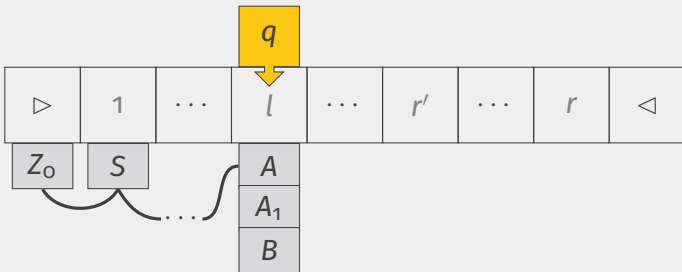
# PEG $\rightarrow$ DPPDA (PART OF CONSTRUCTION)

$$A \leftarrow BC$$



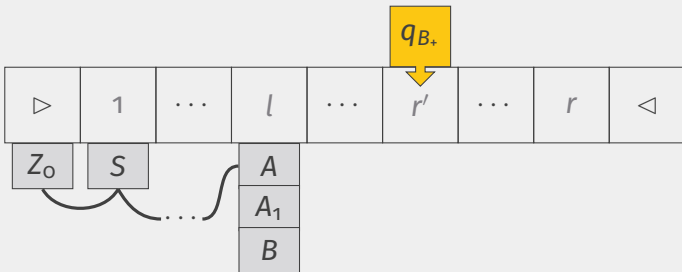
# PEG $\rightarrow$ DPPDA (PART OF CONSTRUCTION)

$$A \leftarrow BC$$



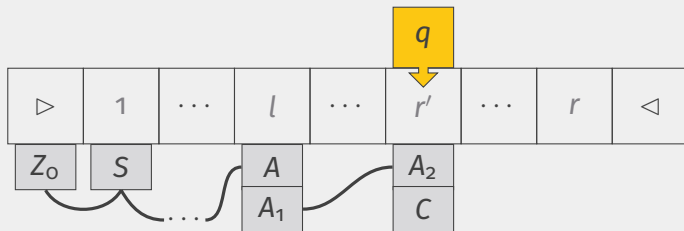
# PEG $\rightarrow$ DPPDA (PART OF CONSTRUCTION)

$$A \leftarrow BC$$



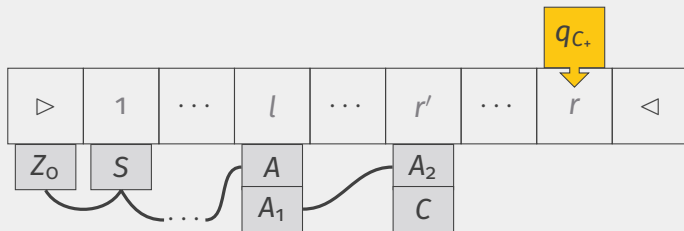
# PEG $\rightarrow$ DPPDA (PART OF CONSTRUCTION)

$$A \leftarrow BC$$



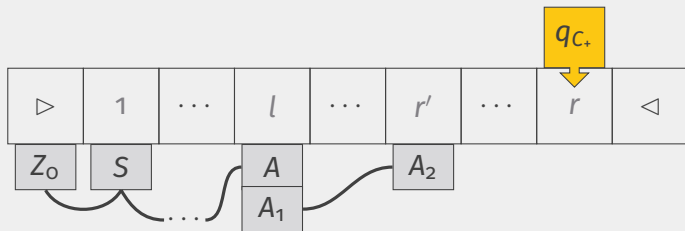
# PEG $\rightarrow$ DPPDA (PART OF CONSTRUCTION)

$$A \leftarrow BC$$



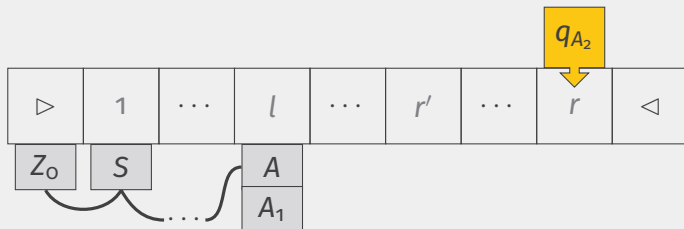
# PEG $\rightarrow$ DPPDA (PART OF CONSTRUCTION)

$$A \leftarrow BC$$



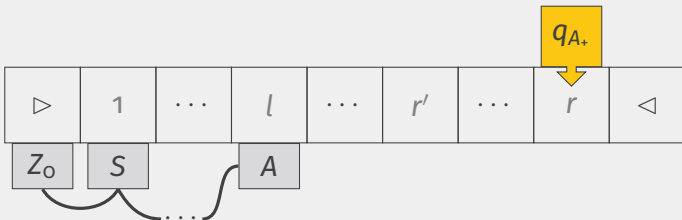
# PEG $\rightarrow$ DPPDA (PART OF CONSTRUCTION)

$$A \leftarrow BC$$



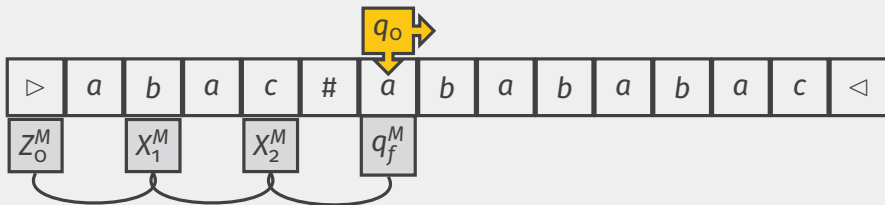
# PEG $\rightarrow$ DPPDA (PART OF CONSTRUCTION)

$$A \leftarrow BC$$

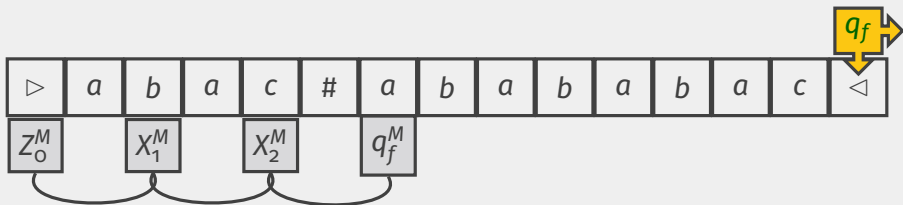




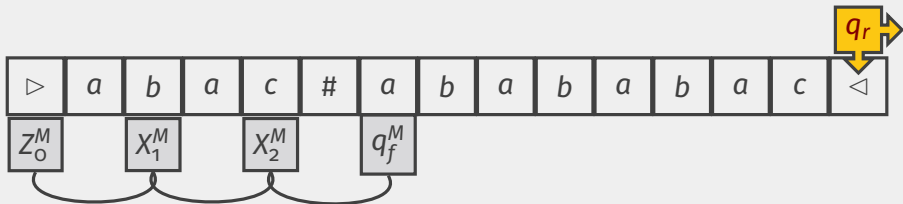
$$\text{DCFL} \cdot \text{PEL} = \text{PEL}$$



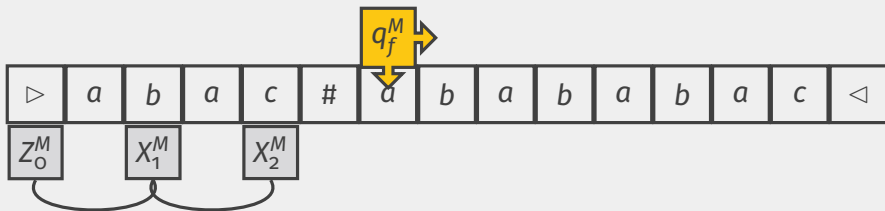
$$\text{DCFL} \cdot \text{PEL} = \text{PEL}$$



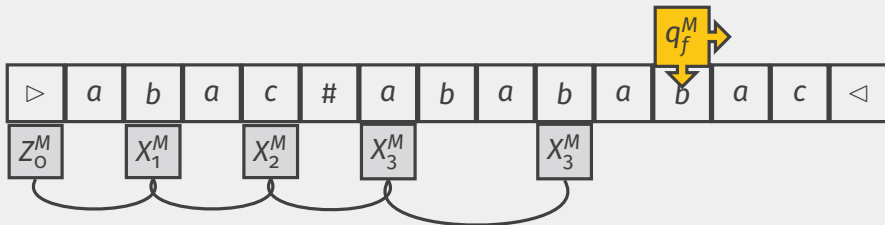
$$\text{DCFL} \cdot \text{PEL} = \text{PEL}$$



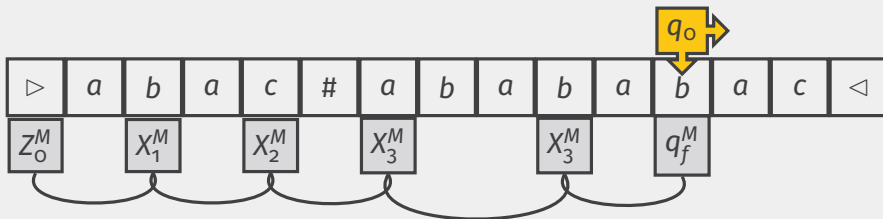
$$\text{DCFL} \cdot \text{PEL} = \text{PEL}$$



$$\text{DCFL} \cdot \text{PEL} = \text{PEL}$$



$$\text{DCFL} \cdot \text{PEL} = \text{PEL}$$



## Theorem

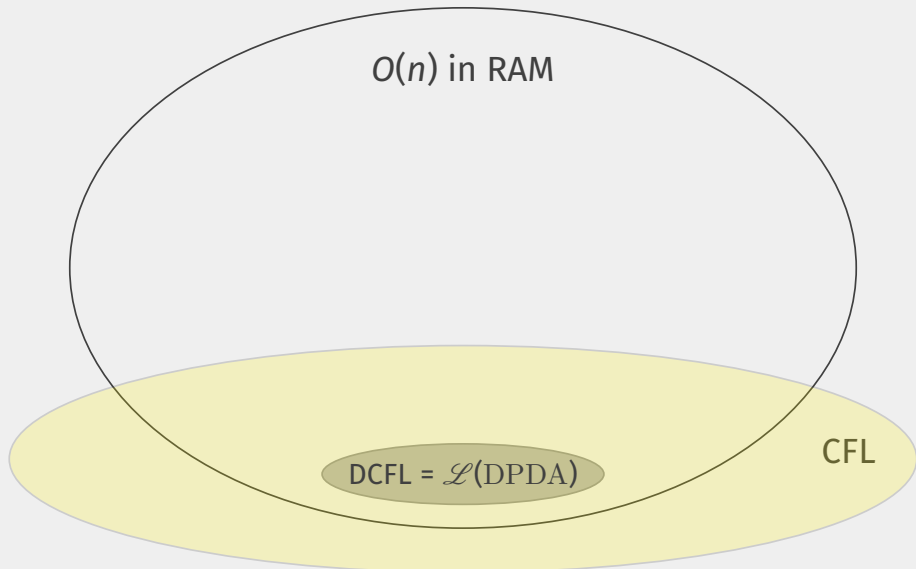
$$\Gamma_{\text{REG}}(\text{DCFL}) \cdot \text{PEL} = \text{PEL}$$

$$\psi : a(a|bc^*)^*b \mapsto L_a(L_a|L_bL_c^*)^*L_b, \text{ where } L_a, L_b, L_c \in \text{DCFL}$$

$$\Gamma_{\text{REG}}(\text{DCFL}) = \bigcup_{\psi, R} \psi(R)$$

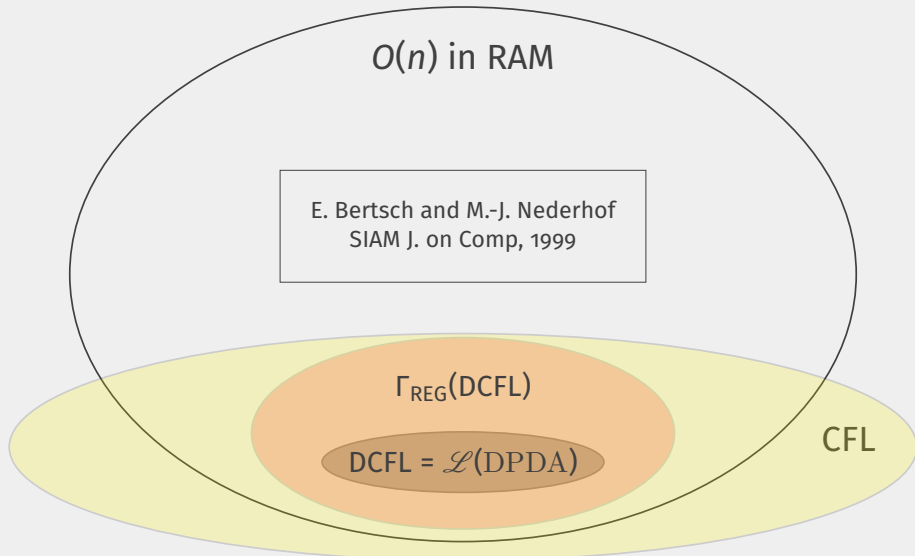
# **OVERVIEW OF RESULTS ON COMPUTATIONAL COMPLEXITY**

# RESULTS ON COMPLEXITY [1/2]

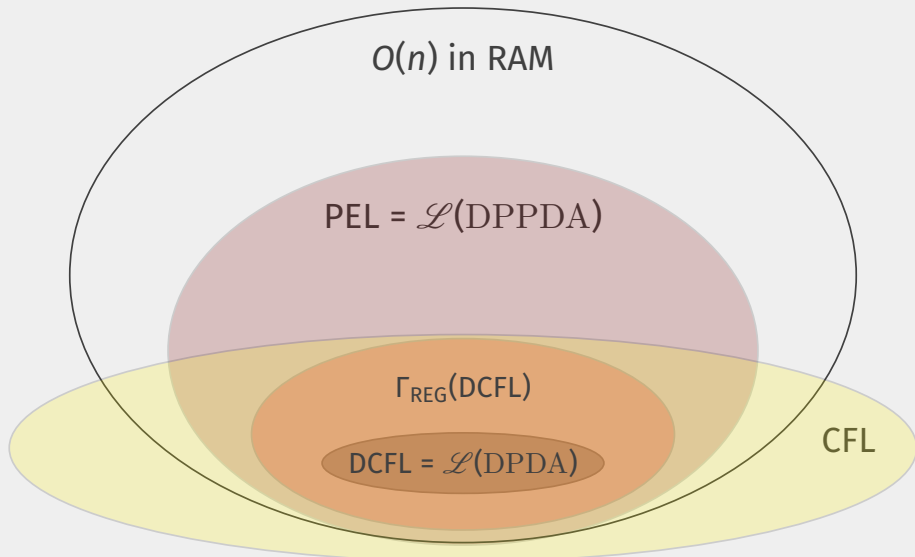




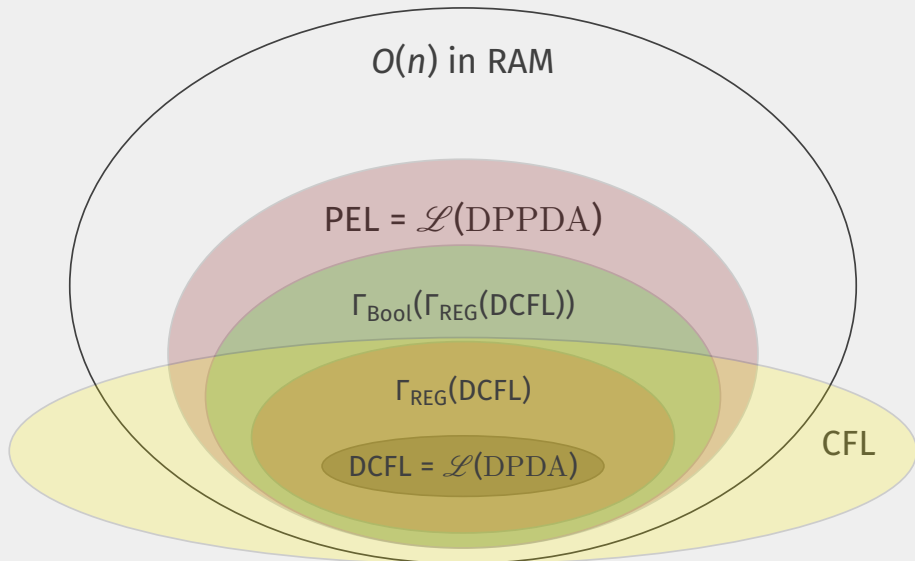
# RESULTS ON COMPLEXITY [1/2]



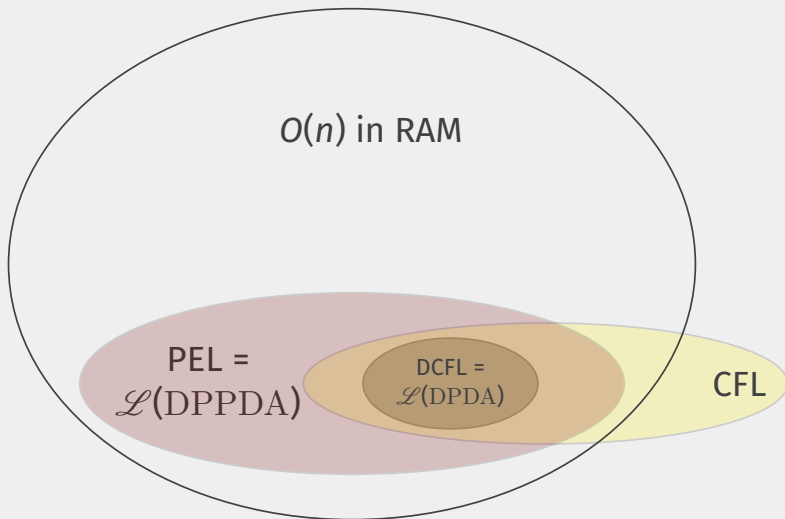
# RESULTS ON COMPLEXITY [1/2]



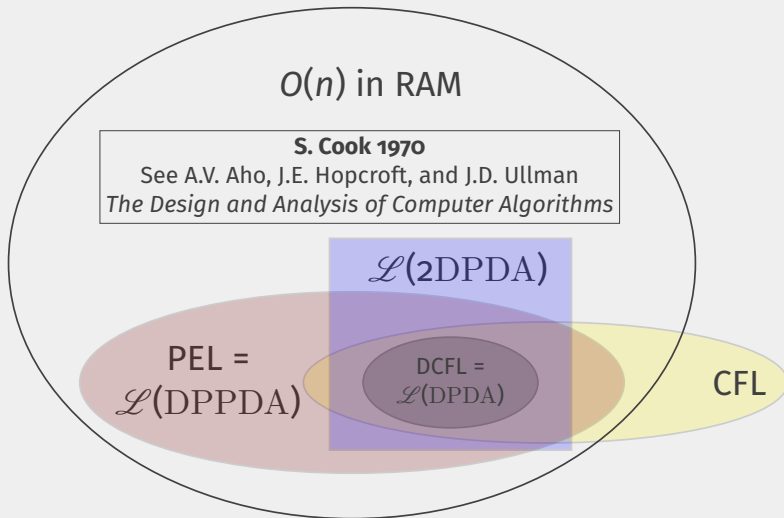
# RESULTS ON COMPLEXITY [1/2]



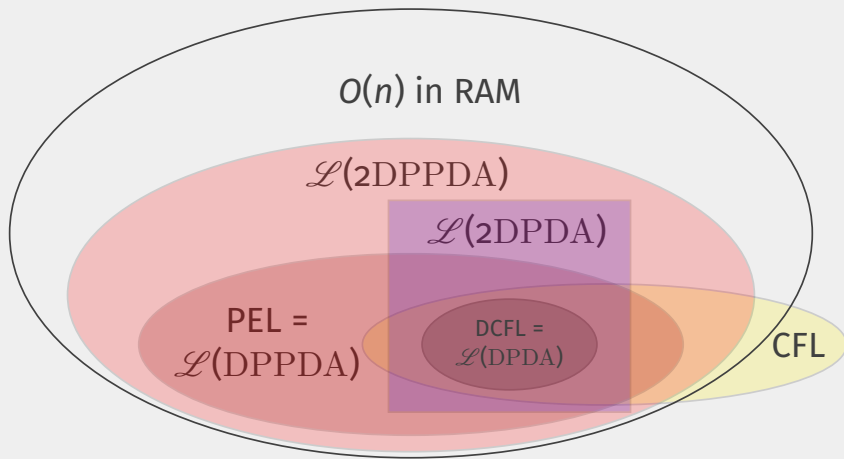
# RESULTS ON COMPLEXITY [2/2]



# RESULTS ON COMPLEXITY [2/2]



# RESULTS ON COMPLEXITY [2/2]



# **RESULTS AND CONCLUSION**

# OVERVIEW OF PREVIOUS RESULTS

- PEGs is an upgrade of Top-Down Parsing Languages (TDPLs) introduced by A. Birman and J. Ullman in the 1960-s
  - ▶  $\text{DCFL} \subseteq \text{TDPL}$ ,  $a^n b^n c^n \in \text{TDPL}$
  - ▶ Linear-time parsing algorithm, which was impractical in 60-s, because of memory overhead



# OVERVIEW OF PREVIOUS RESULTS

- PEGs is an upgrade of Top-Down Parsing Languages (TDPLs) introduced by A. Birman and J. Ullman in the 1960-s
  - ▶  $DCFL \subseteq TDPL$ ,  $a^n b^n c^n \in TDPL$
  - ▶ Linear-time parsing algorithm, which was impractical in 60-s, because of memory overhead
- B. Ford invented PEGs in 2004
  - ▶ PEGs are equivalent to TDPLs and generalized TDPLs
  - ▶ Moreover PEGs without ! operation generate PEL
  - ▶ Practical linear-time parsing algorithm for PEGs (Packrat Parsing)

# OVERVIEW OF PREVIOUS RESULTS

- PEGs is an upgrade of Top-Down Parsing Languages (TDPLs) introduced by A. Birman and J. Ullman in the 1960-s
  - ▶  $DCFL \subseteq TDPL$ ,  $a^n b^n c^n \in TDPL$
  - ▶ Linear-time parsing algorithm, which was impractical in 60-s, because of memory overhead
- B. Ford invented PEGs in 2004
  - ▶ PEGs are equivalent to TDPLs and generalized TDPLs
  - ▶ Moreover PEGs without ! operation generate PEL
  - ▶ Practical linear-time parsing algorithm for PEGs (Packrat Parsing)
- Python replaced LL-parser by PEG (PEP 617, 2020)

# OVERVIEW OF PREVIOUS RESULTS

- PEGs is an upgrade of Top-Down Parsing Languages (TDPLs) introduced by A. Birman and J. Ullman in the 1960-s
  - ▶  $DCFL \subseteq TDPL$ ,  $a^n b^n c^n \in TDPL$
  - ▶ Linear-time parsing algorithm, which was impractical in 60-s, because of memory overhead
- B. Ford invented PEGs in 2004
  - ▶ PEGs are equivalent to TDPLs and generalized TDPLs
  - ▶ Moreover PEGs without ! operation generate PEL
  - ▶ Practical linear-time parsing algorithm for PEGs (Packrat Parsing)
- Python replaced LL-parser by PEG (PEP 617, 2020)
- B. Loff, N. Moreira, and R. Reis presented the first computational model for PEGs (DLT, 2018)
  - ▶  $a^{2^n} \in PEL$  and palindromes of length  $2^n$  in PEL
  - ▶ Structural Results: there is no pumping lemma for PEL

# OUR RESULTS

- New computational model: Pushdown Pointer Automata
  - ▶  $\mathcal{L}(\text{DPPDA}) = \text{PEL}$
  - ▶ Linear-time simulation algorithm for 2-DPPDA (in RAM), modification of S. Cook algorithm for 2-DPDA
  - ▶ Clarification of PEL place among the formal languages
  - ▶ Simplicity: now the inclusion  $\text{DCFL} \subseteq \text{PEL}$  is trivial
- **Thm.**  $\Gamma_{\text{REG}}(\text{DCFL}) \cdot \text{PEL} = \text{PEL}$ 
  - ▶ **Corollary:**  $\Gamma_{\text{REG}}(\text{DCFL}) \in \text{PEL} \Rightarrow \Gamma_{\text{Bool}}(\Gamma_{\text{REG}}(\text{DCFL})) \in \text{PEL} \Rightarrow \Gamma_{\text{Bool}}(\Gamma_{\text{REG}}(\text{DCFL}))$  is  $O(n)$ -recognizable in RAM.
    - It is a simplification and upgrade of the previously known result:  $\Gamma_{\text{REG}}(\text{DCFL})$  is  $O(n)$ -recognizable [E. Bertsch and M.-J. Nederhof, SIAM J. on Comp, 1999]
- We hope that our model will raise interest to PELs in TCS community

# DISCUSSION

- The relations between following models are unknown:
  - ▶ DPPDA vs 2-DPDA
  - ▶ 2-DPDA vs 2-DPPDA
  - ▶ 2-DPDA vs 2-NPDA
  - ▶ 2-NPDA vs 2-NPPDA
- It is unknown whether  $\text{CFL} \stackrel{?}{\subseteq} \mathcal{L}(2\text{-DPPDA})$

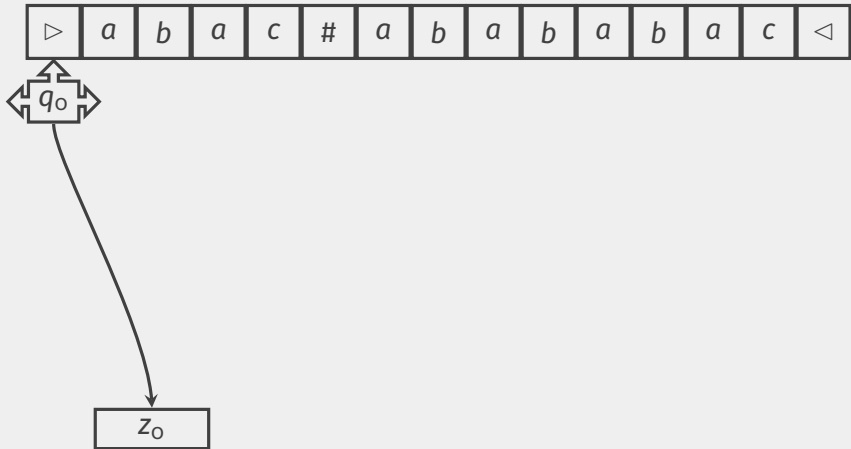
## News

Now practically-motivated class PEL is among these classes

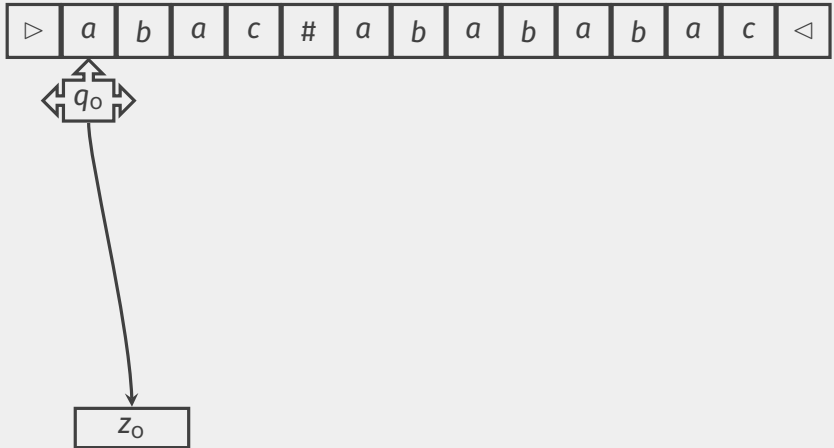
## Remark

Many questions for 2-NDPA from Rupak Majumdar's talk are relevant for 2-NPPDA

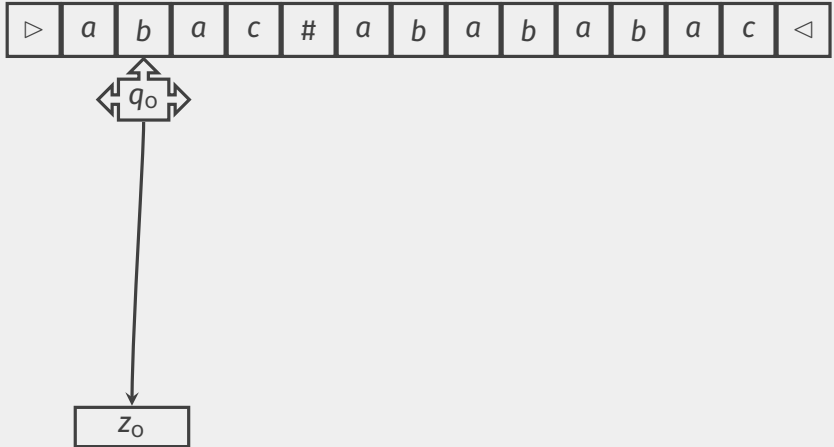
# KMP-ALGORITHM + 2DPDA



# KMP-ALGORITHM + 2DPDA

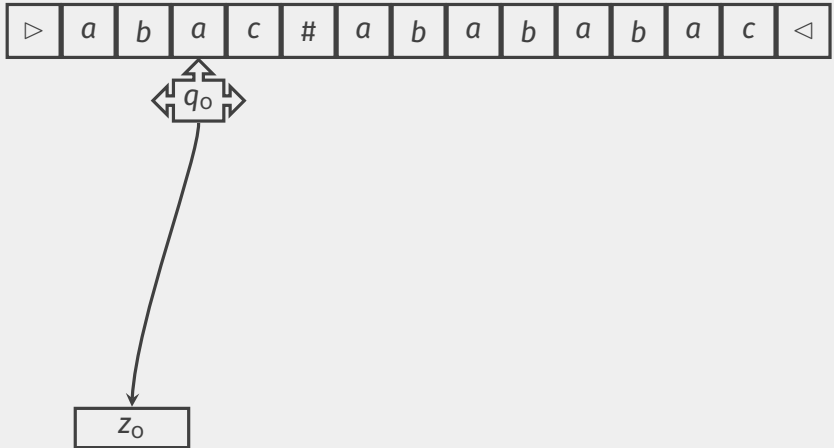


# KMP-ALGORITHM + 2DPDA

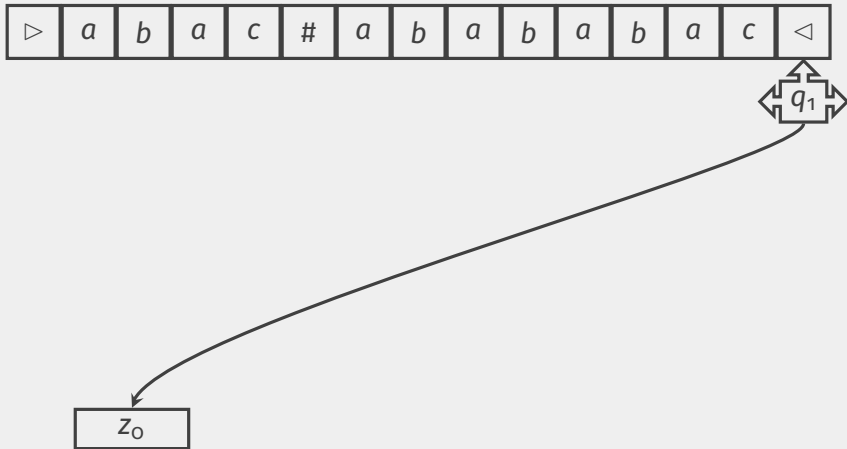




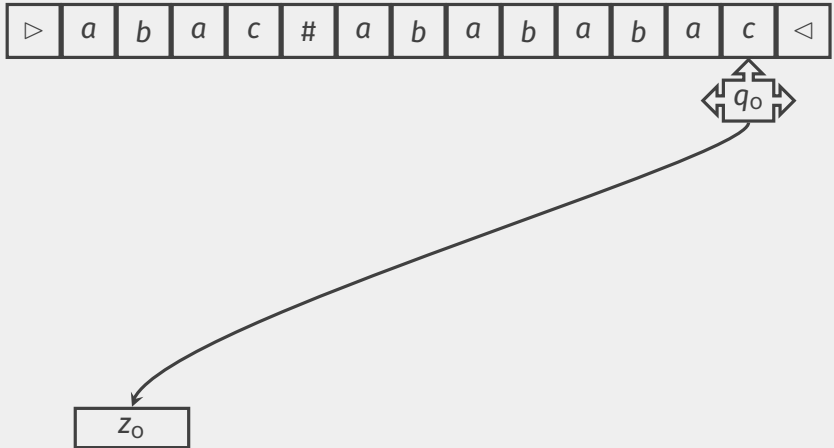
# KMP-ALGORITHM + 2DPDA



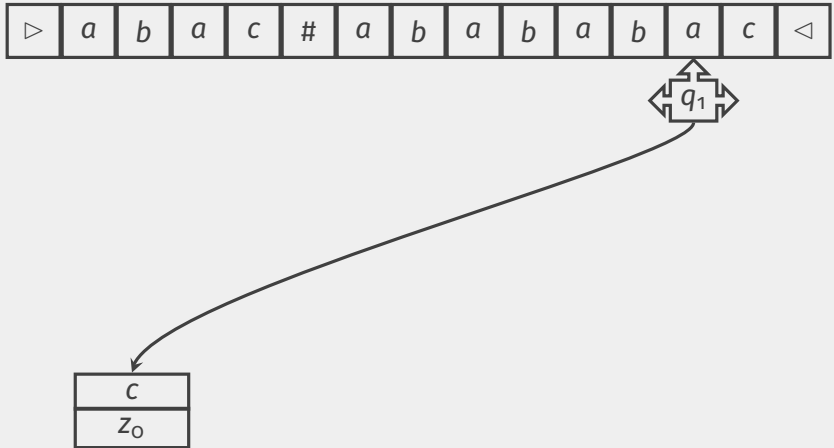
# KMP-ALGORITHM + 2DPDA



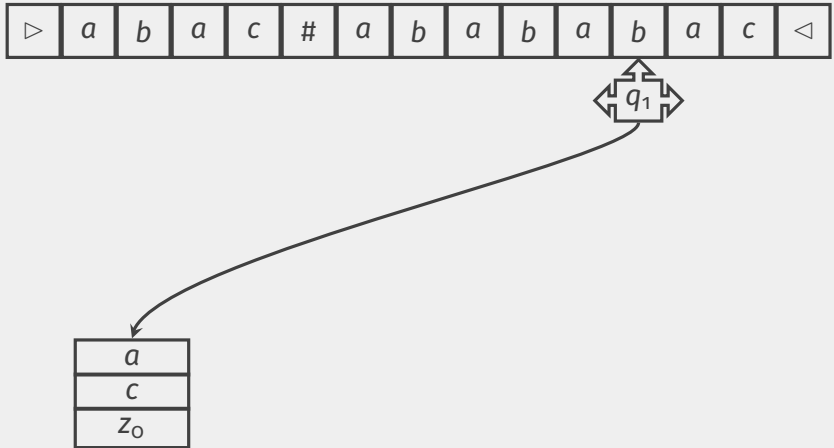
# KMP-ALGORITHM + 2DPDA



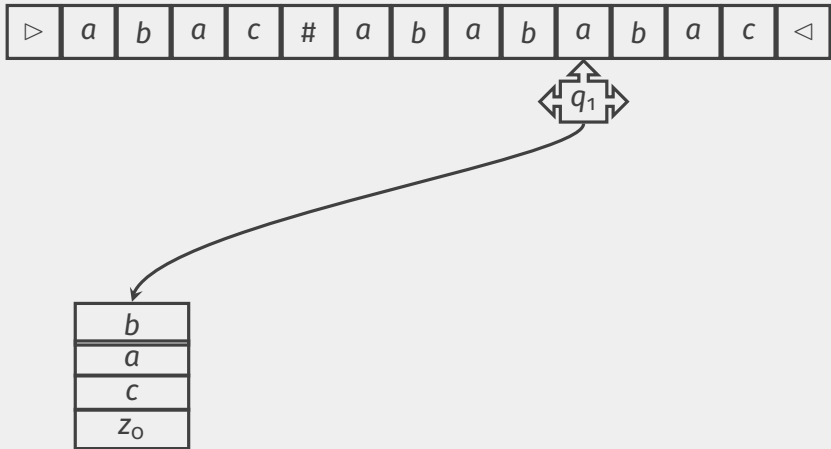
# KMP-ALGORITHM + 2DPDA



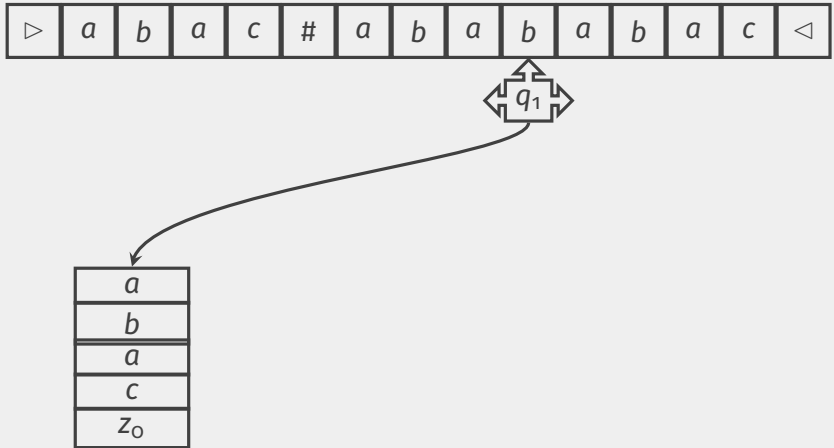
# KMP-ALGORITHM + 2DPDA



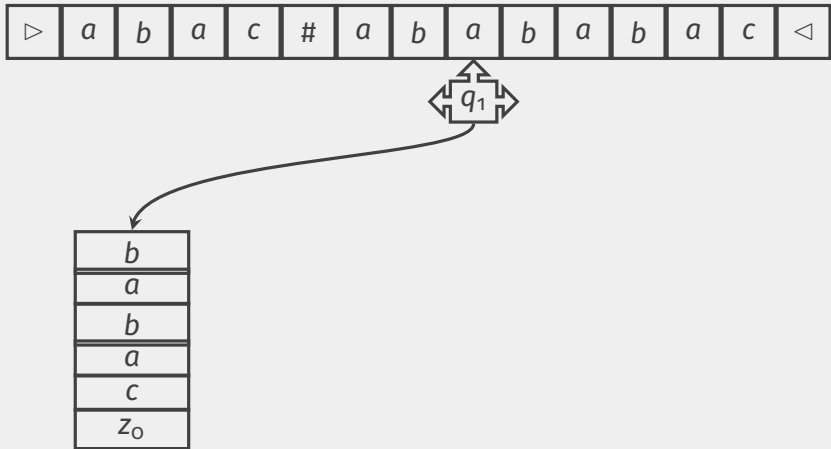
# KMP-ALGORITHM + 2DPDA



# KMP-ALGORITHM + 2DPDA

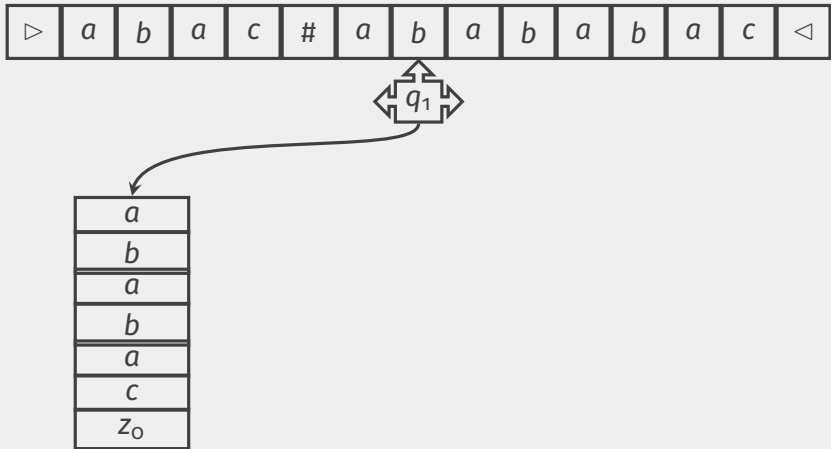


# KMP-ALGORITHM + 2DPDA

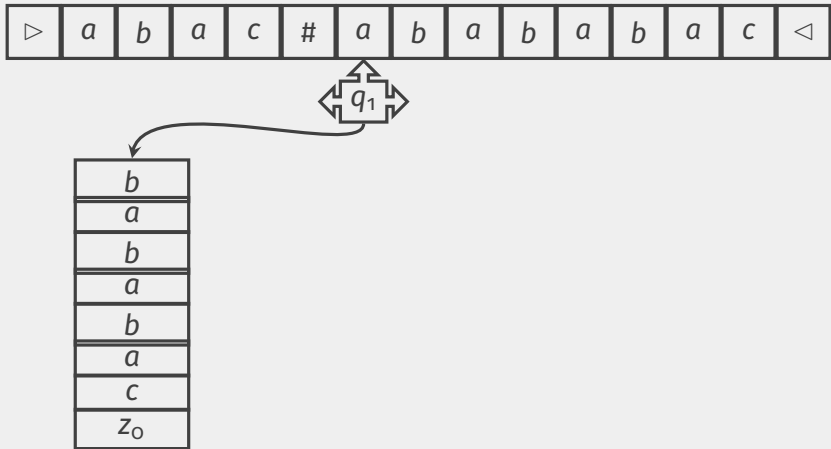




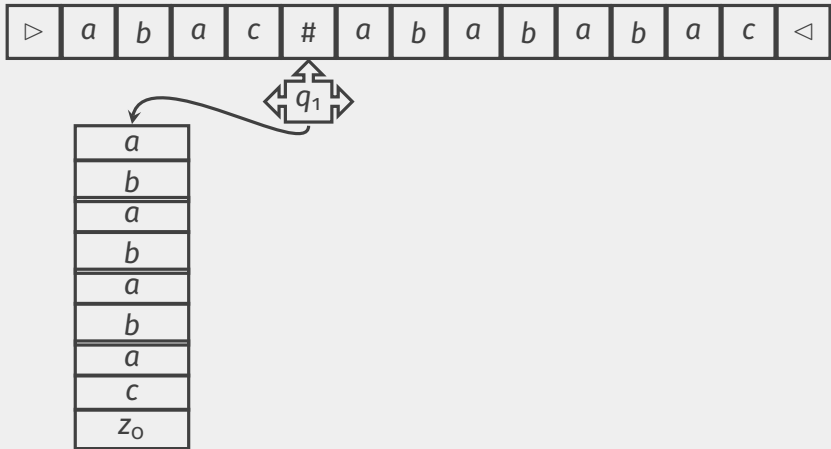
# KMP-ALGORITHM + 2DPDA



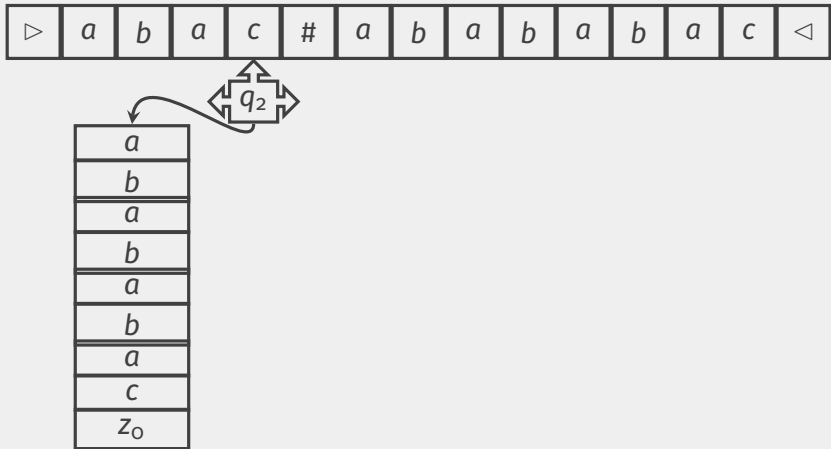
# KMP-ALGORITHM + 2DPDA



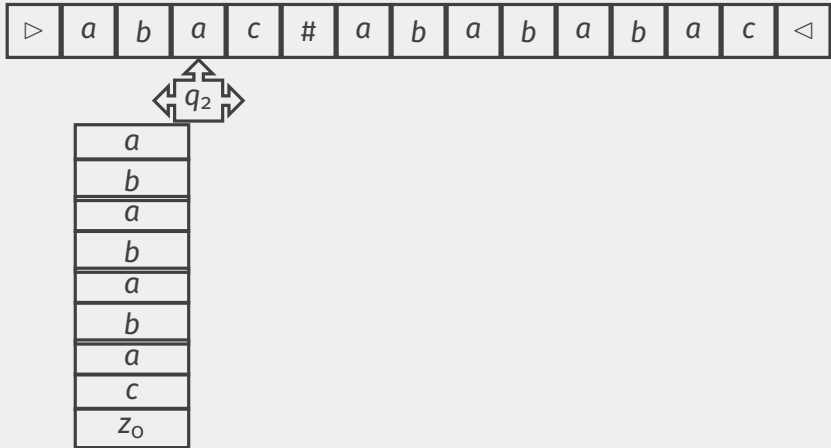
# KMP-ALGORITHM + 2DPDA



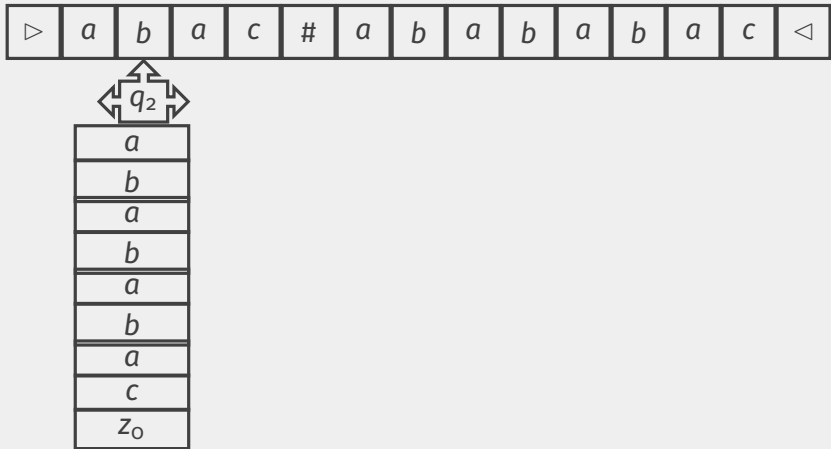
# KMP-ALGORITHM + 2DPDA



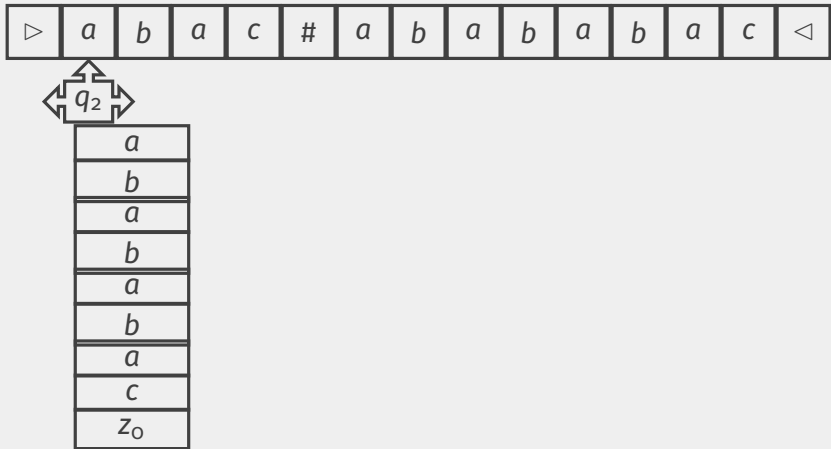
# KMP-ALGORITHM + 2DPDA



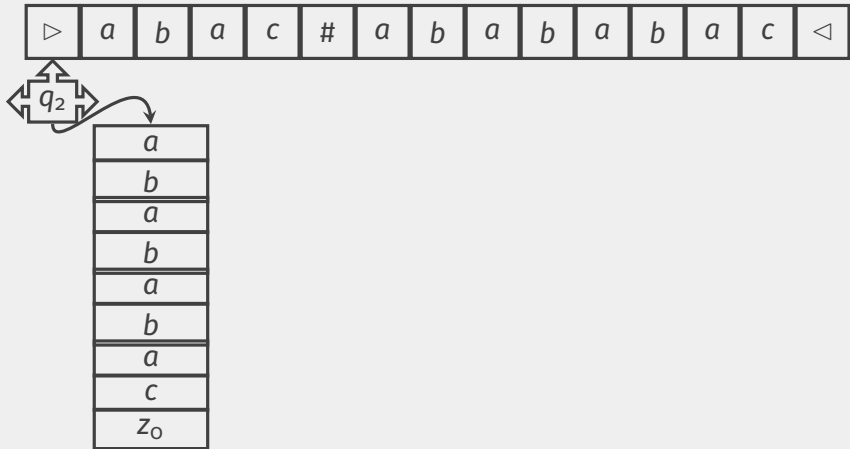
# KMP-ALGORITHM + 2DPDA



# KMP-ALGORITHM + 2DPDA

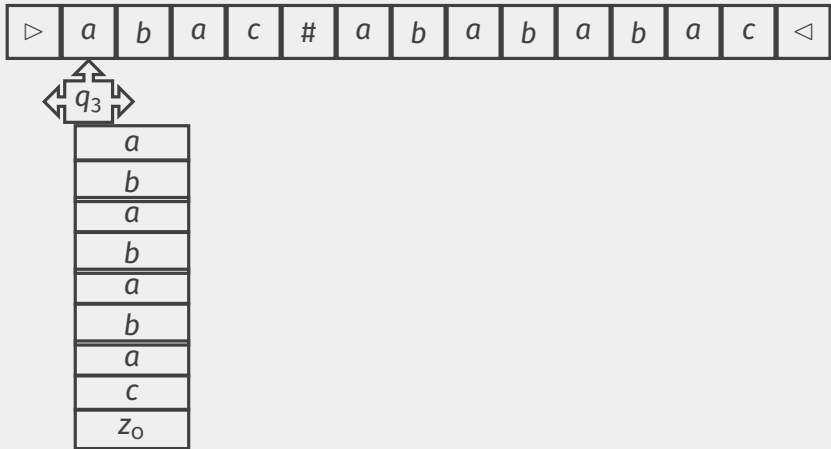


# KMP-ALGORITHM + 2DPDA

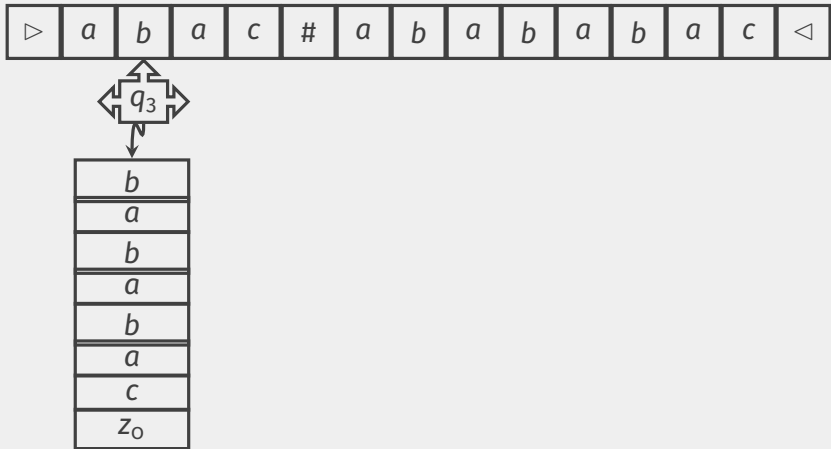




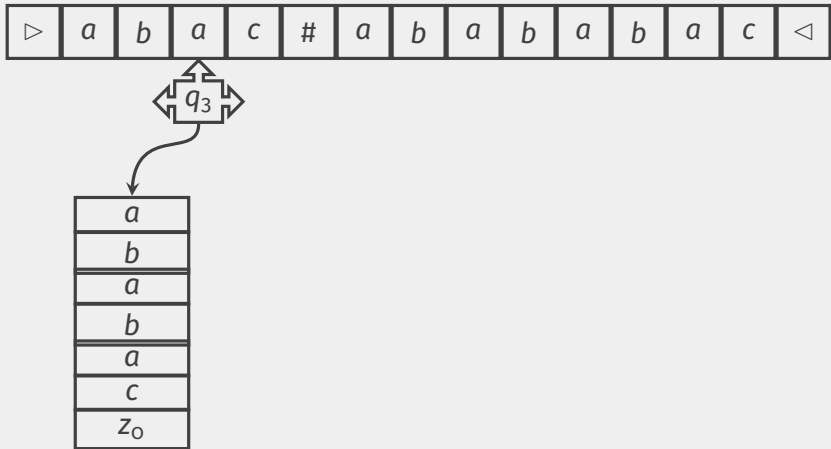
# KMP-ALGORITHM + 2DPDA



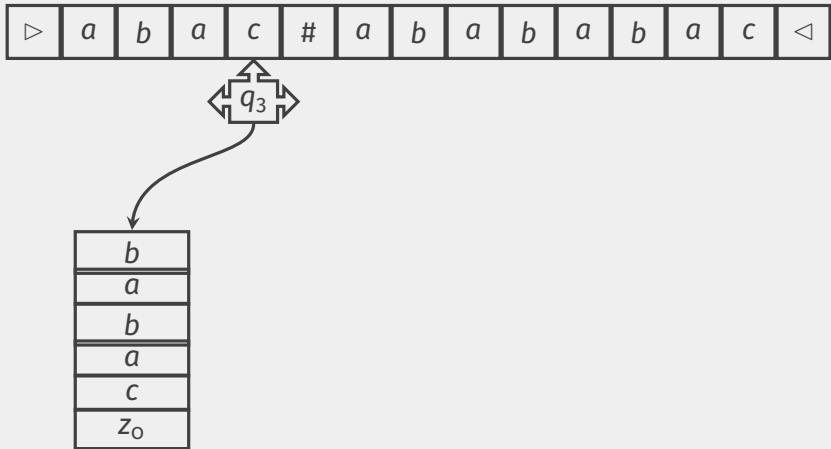
# KMP-ALGORITHM + 2DPDA



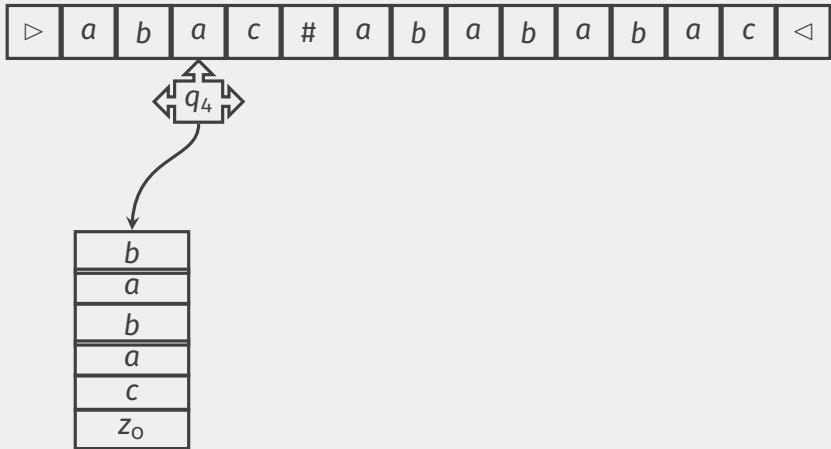
# KMP-ALGORITHM + 2DPDA



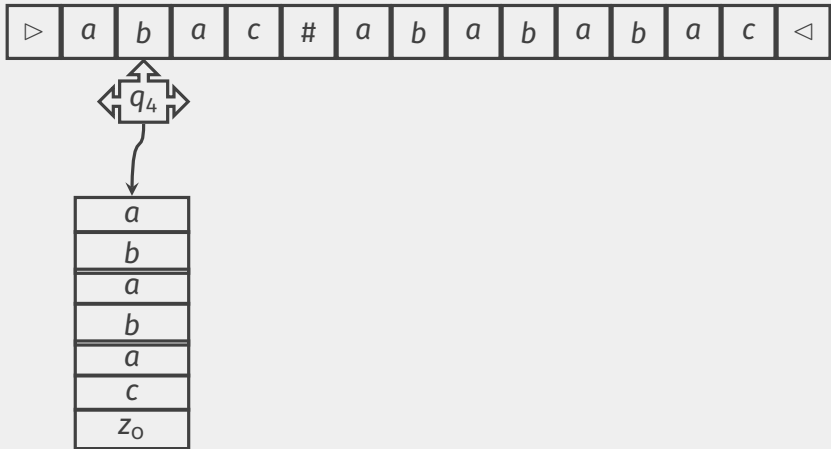
# KMP-ALGORITHM + 2DPDA



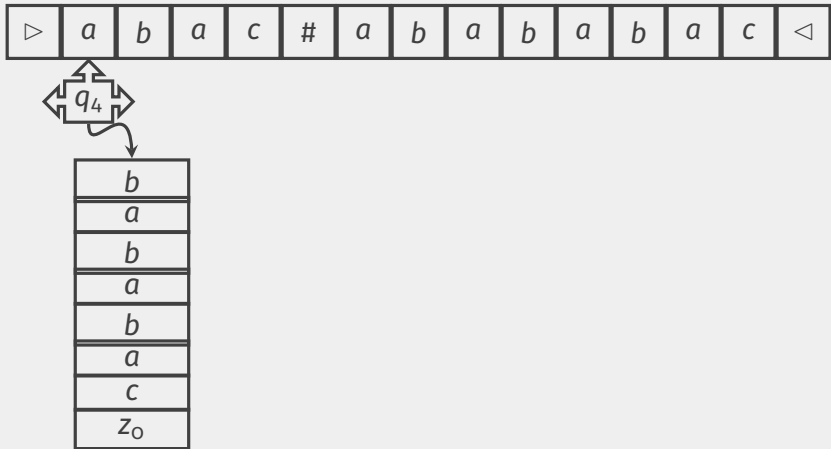
# KMP-ALGORITHM + 2DPDA



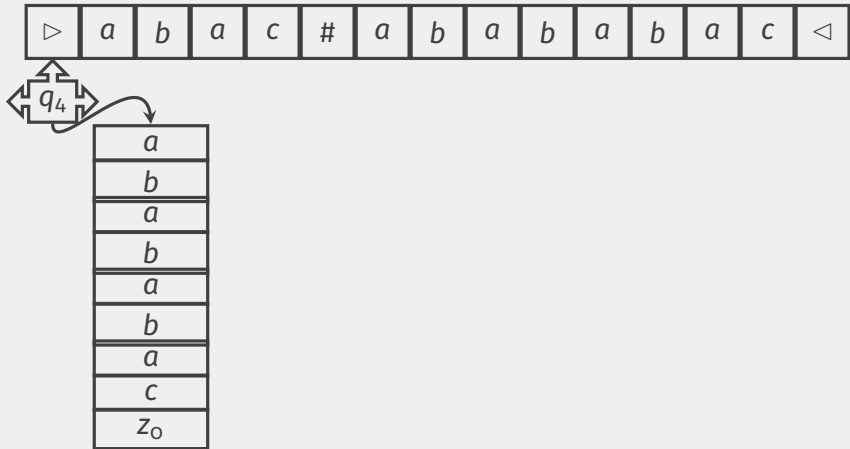
# KMP-ALGORITHM + 2DPDA



# KMP-ALGORITHM + 2DPDA

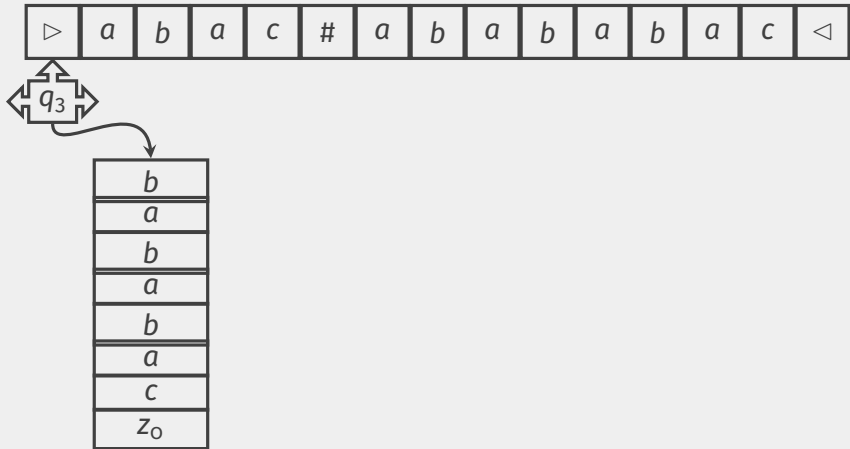


# KMP-ALGORITHM + 2DPDA

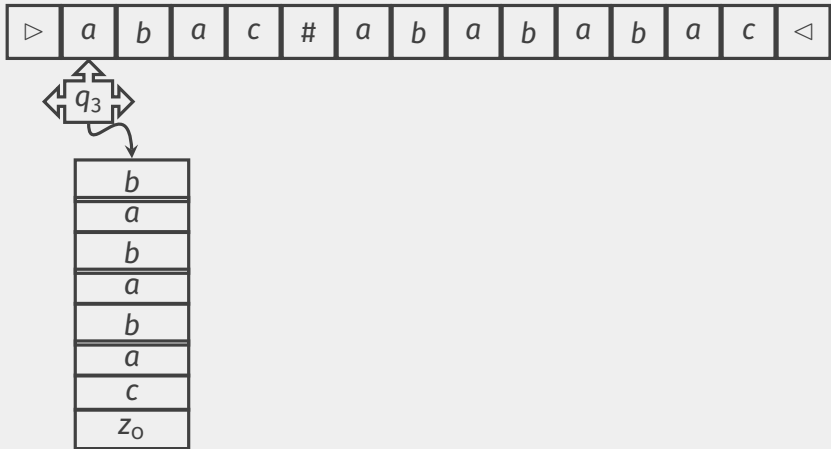




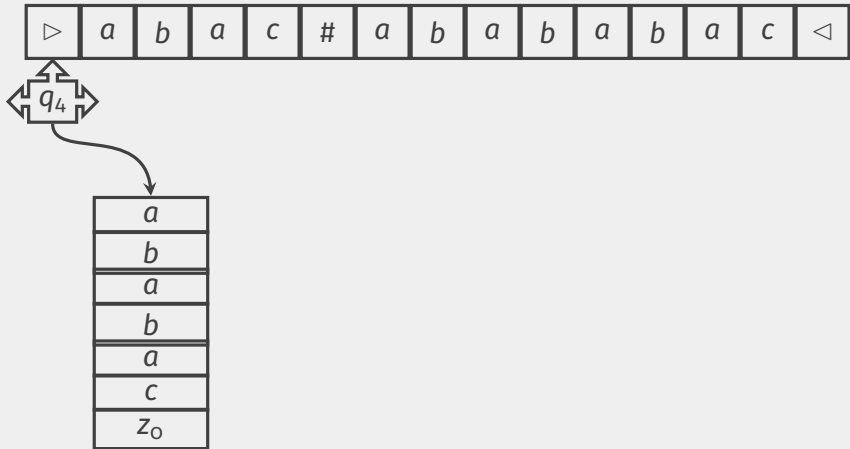
# KMP-ALGORITHM + 2DPDA



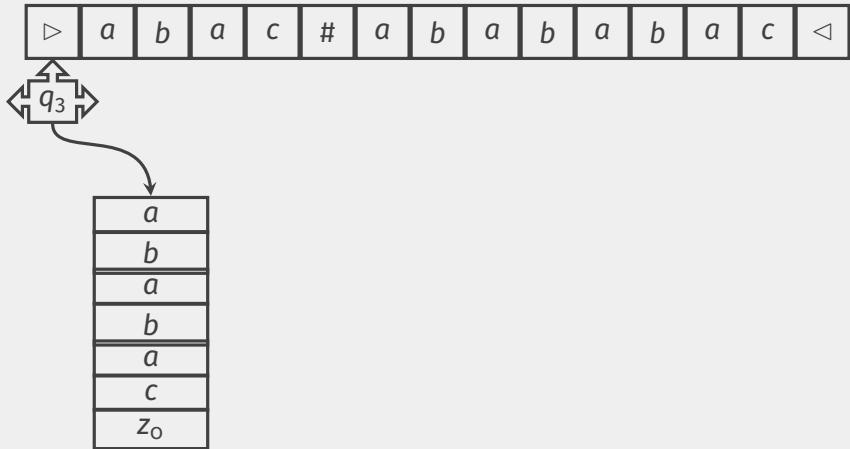
# KMP-ALGORITHM + 2DPDA



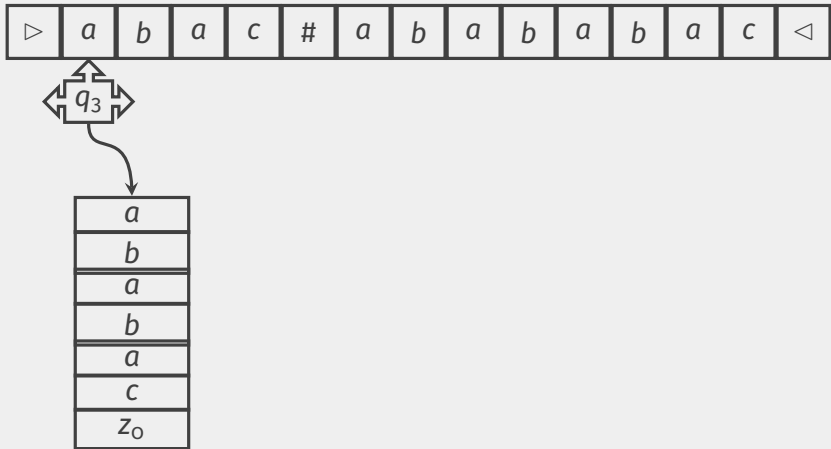
# KMP-ALGORITHM + 2DPDA



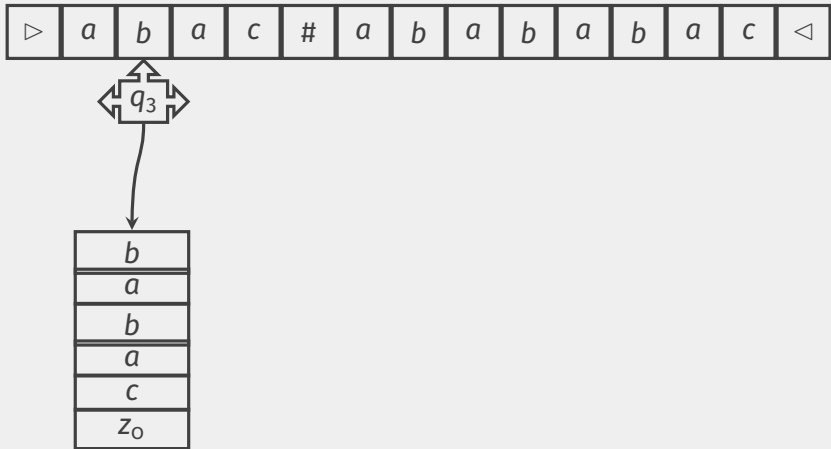
# KMP-ALGORITHM + 2DPDA



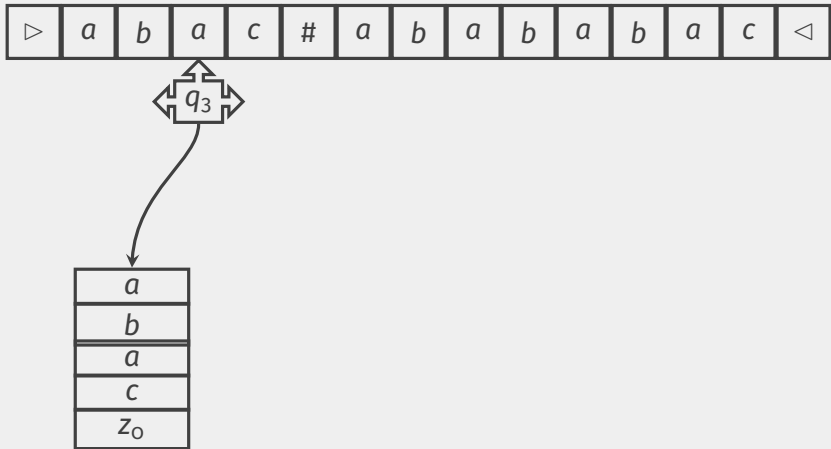
# KMP-ALGORITHM + 2DPDA



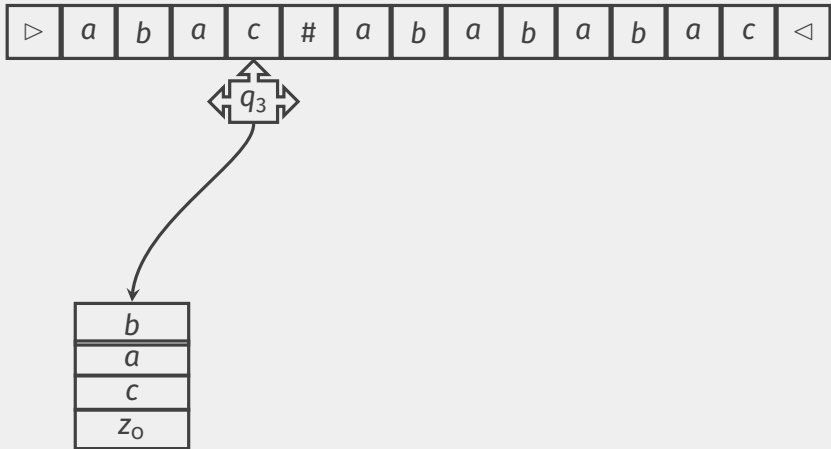
# KMP-ALGORITHM + 2DPDA



# KMP-ALGORITHM + 2DPDA

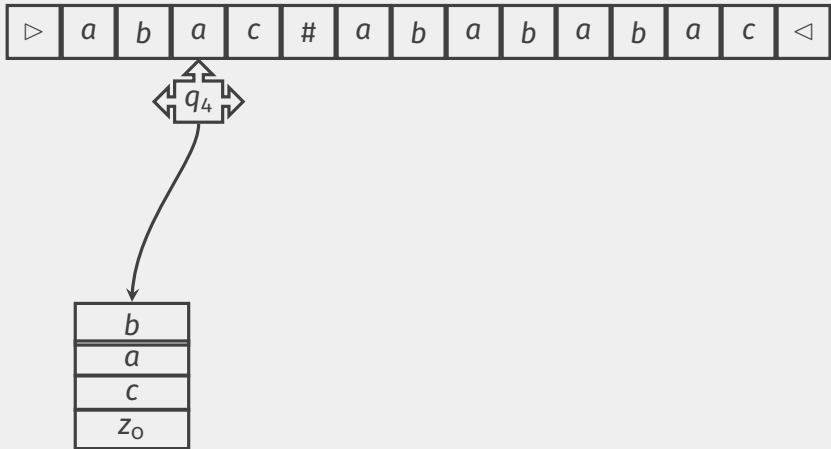


# KMP-ALGORITHM + 2DPDA

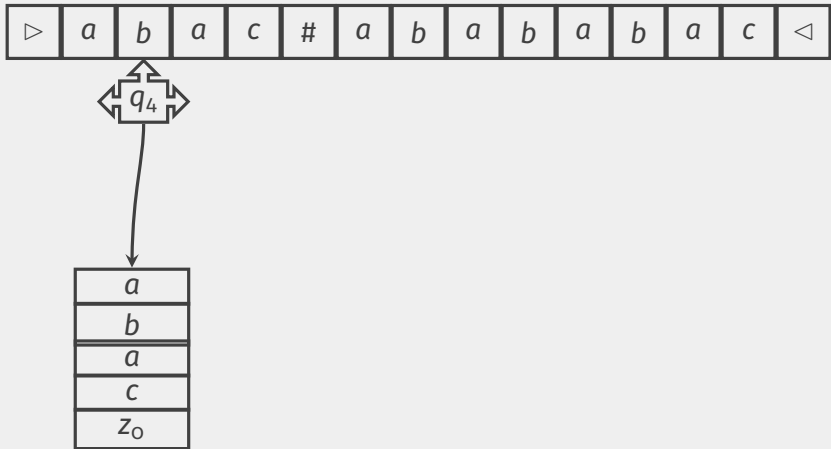




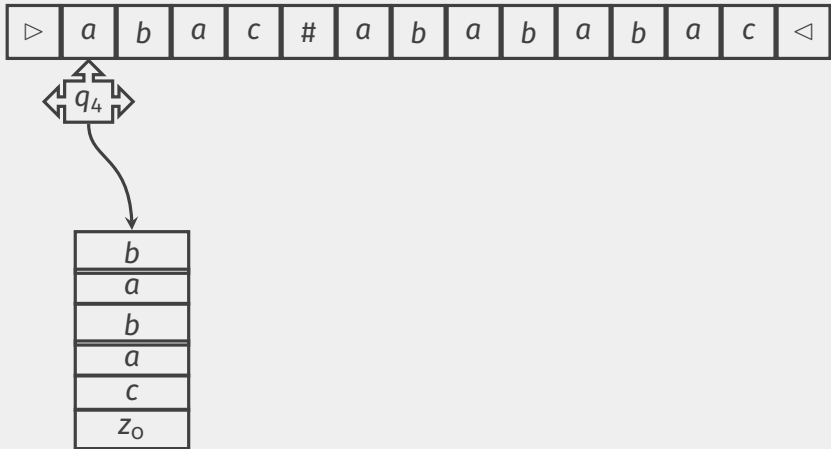
# KMP-ALGORITHM + 2DPDA



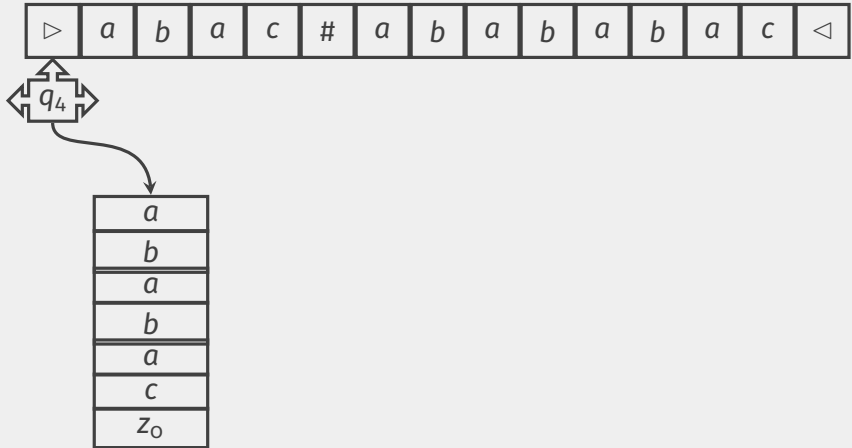
# KMP-ALGORITHM + 2DPDA



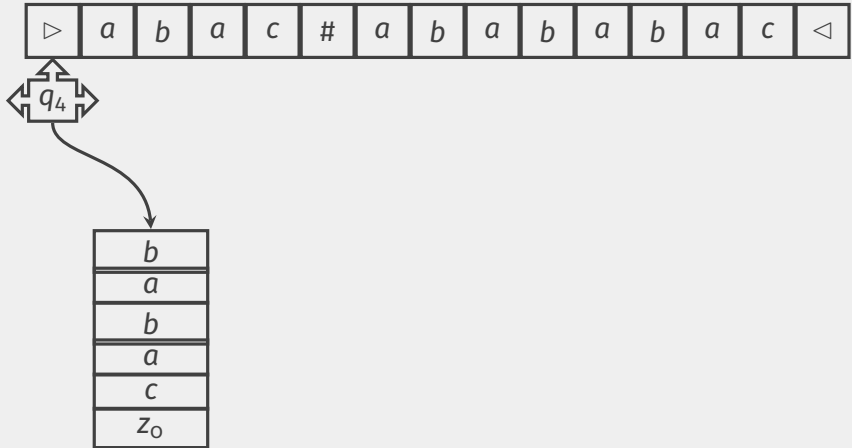
# KMP-ALGORITHM + 2DPDA



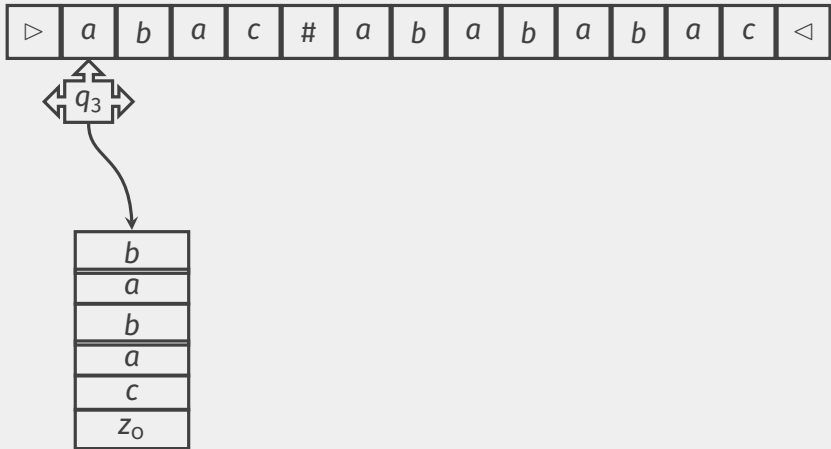
# KMP-ALGORITHM + 2DPDA



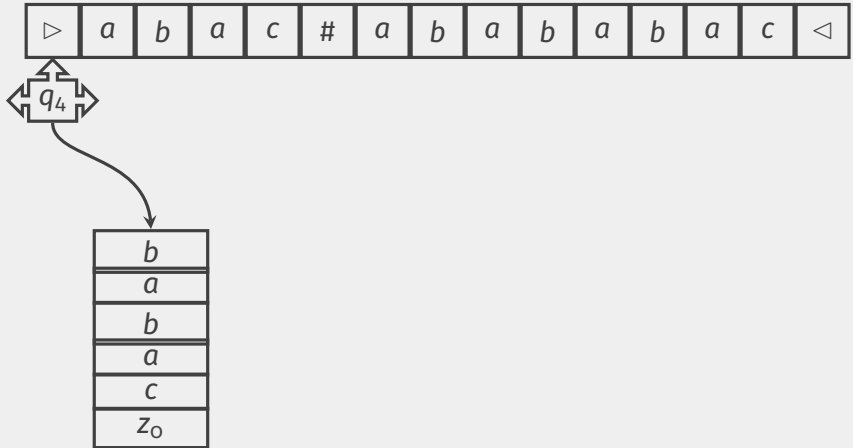
# KMP-ALGORITHM + 2DPDA



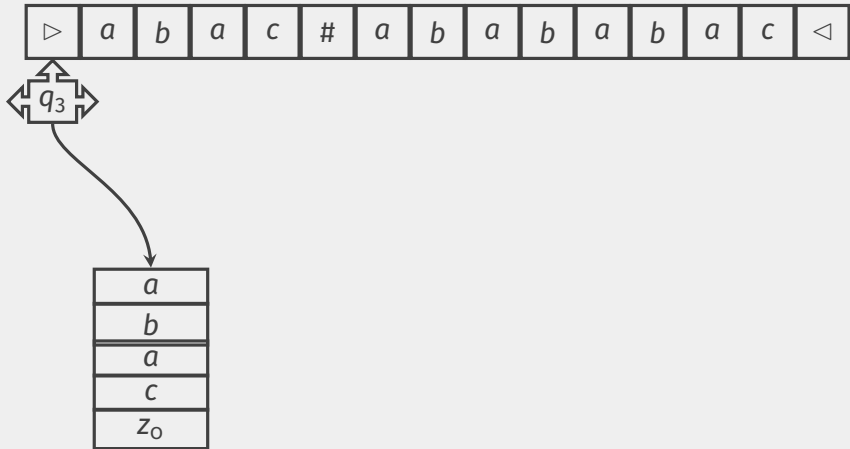
# KMP-ALGORITHM + 2DPDA



# KMP-ALGORITHM + 2DPDA

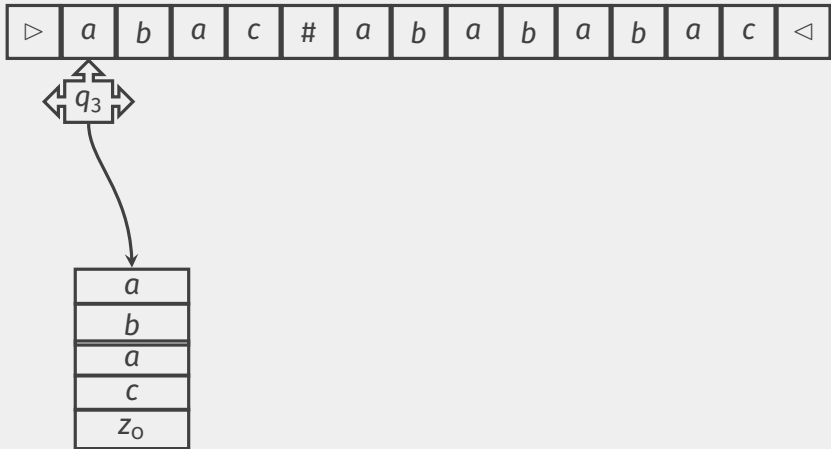


# KMP-ALGORITHM + 2DPDA

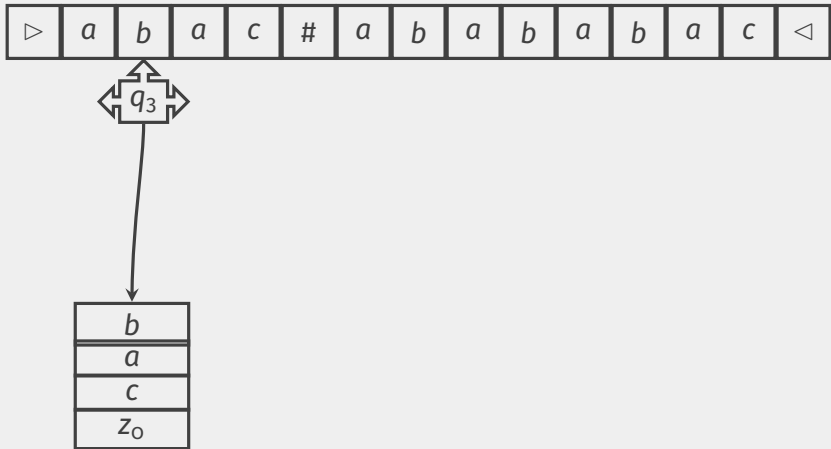




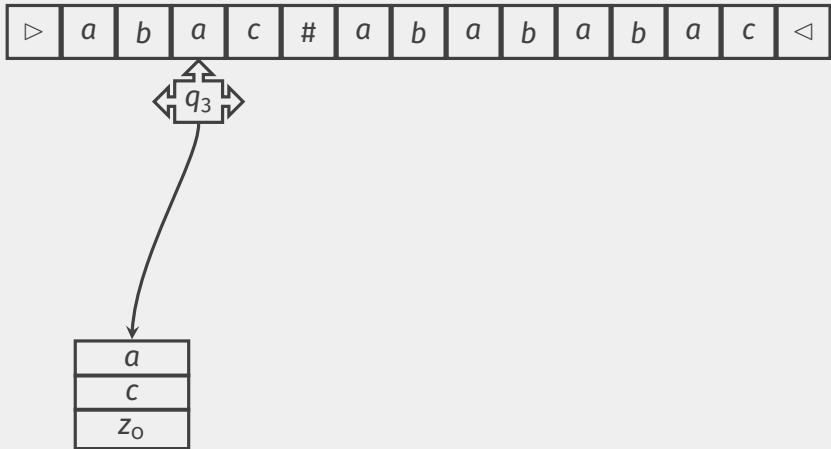
# KMP-ALGORITHM + 2DPDA



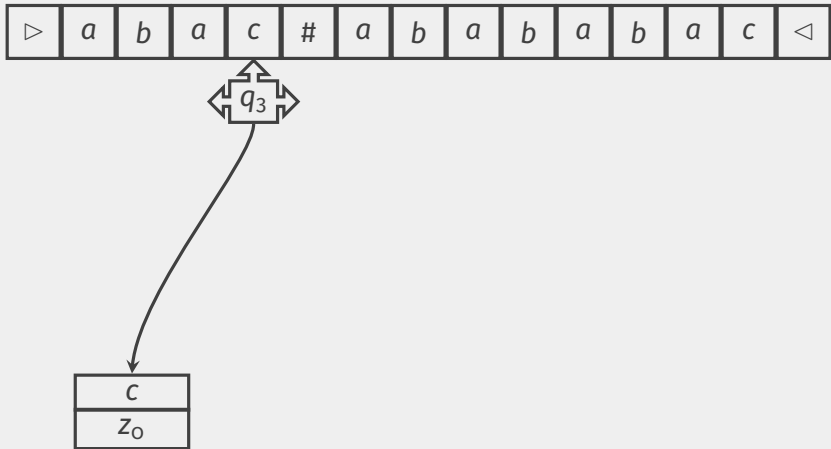
# KMP-ALGORITHM + 2DPDA



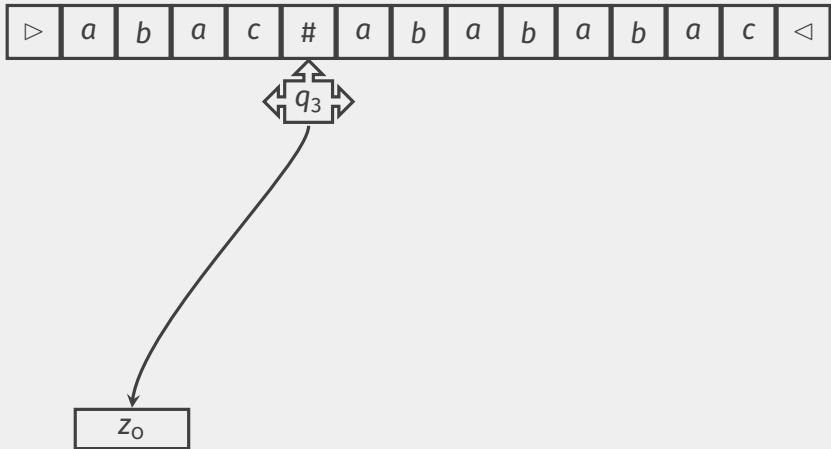
# KMP-ALGORITHM + 2DPDA



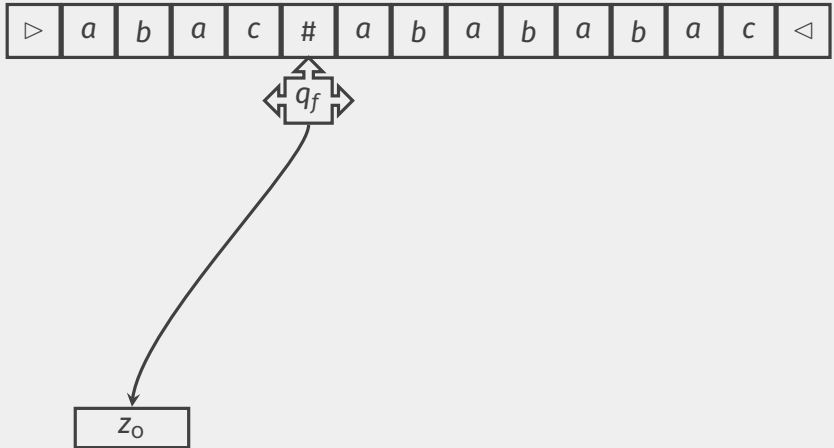
# KMP-ALGORITHM + 2DPDA



# KMP-ALGORITHM + 2DPDA



# KMP-ALGORITHM + 2DPDA



# COOK'S THEOREM

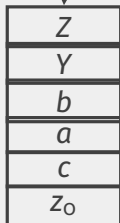
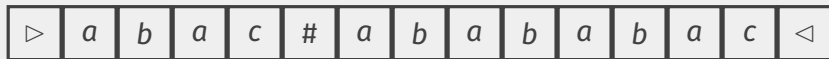
## Theorem (Cook 1972)

*A 2-DPDA-recognizable language is recognizable in linear time (in the RAM model).*

Cook also provided a linear-time simulation algorithm.

- KMP algorithm has been investigated by Knuth by this simulation (and independently discovered by Morris without it).
- LR-parsers are 1-DPDA, so Cook's results show that there is an option of linear time parsing for wider class than DCFL.

# PROOF OF COOK'S THEOREM



**Surface configuration**

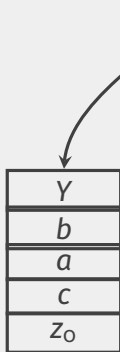
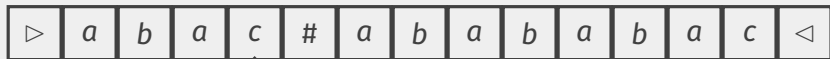
$(4, q, Z)$

- 4 — the head's position
- $q$  — the state
- $Z$  — top stack symbol

**Terminator** for  $(i, q, Z)$  is  $(j, p, Z)$  s.t.  $Z$  is popped.



# PROOF OF COOK'S THEOREM



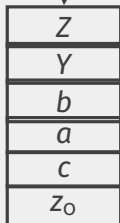
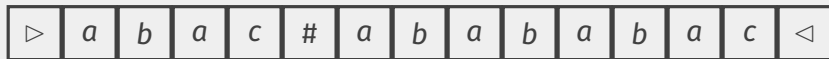
## Surface configuration

$(4, q, Z)$

- 4 — the head's position
- $q$  — the state
- $Z$  — top stack symbol

**Terminator** for  $(i, q, Z)$  is  $(j, p, Z)$  s.t.  $Z$  is popped.

# PROOF OF COOK'S THEOREM



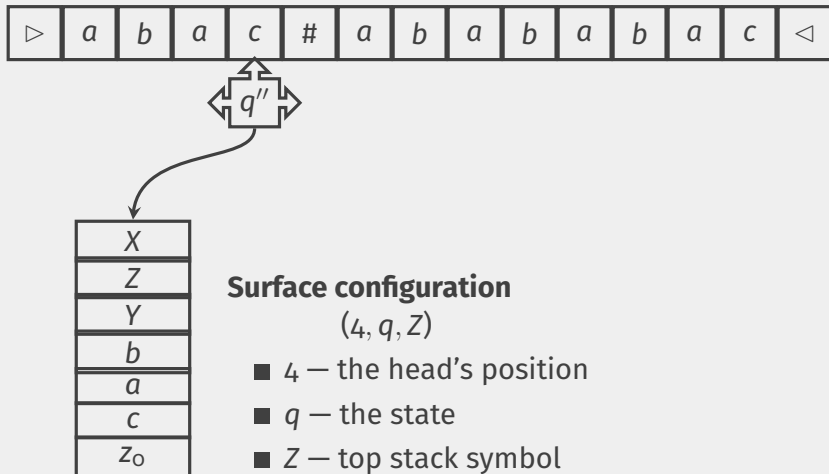
**Surface configuration**

$(4, q, Z)$

- 4 — the head's position
- $q$  — the state
- $Z$  — top stack symbol

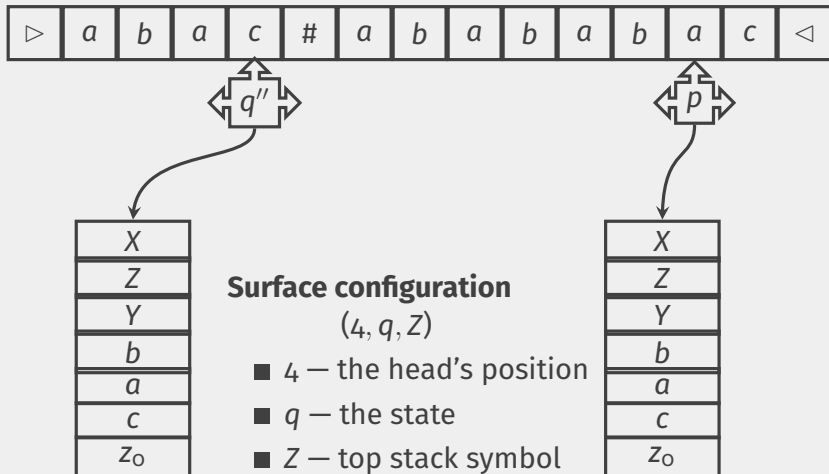
**Terminator** for  $(i, q, Z)$  is  $(j, p, Z)$  s.t.  $Z$  is popped.

# PROOF OF COOK'S THEOREM



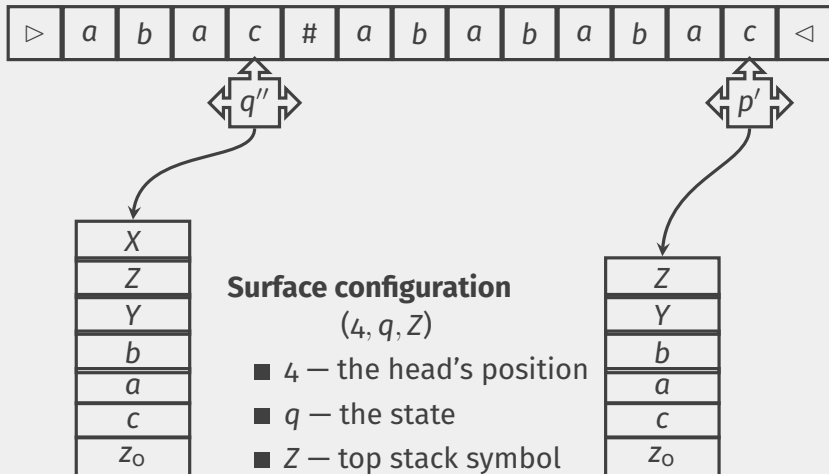
**Terminator** for  $(i, q, Z)$  is  $(j, p, Z)$  s.t.  $Z$  is popped.

# PROOF OF COOK'S THEOREM



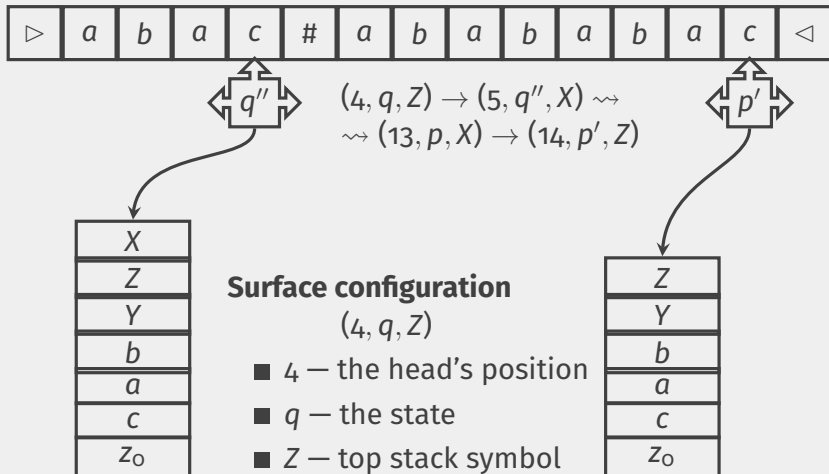
**Terminator** for  $(i, q, Z)$  is  $(j, p, Z)$  s.t.  $Z$  is popped.

# PROOF OF COOK'S THEOREM



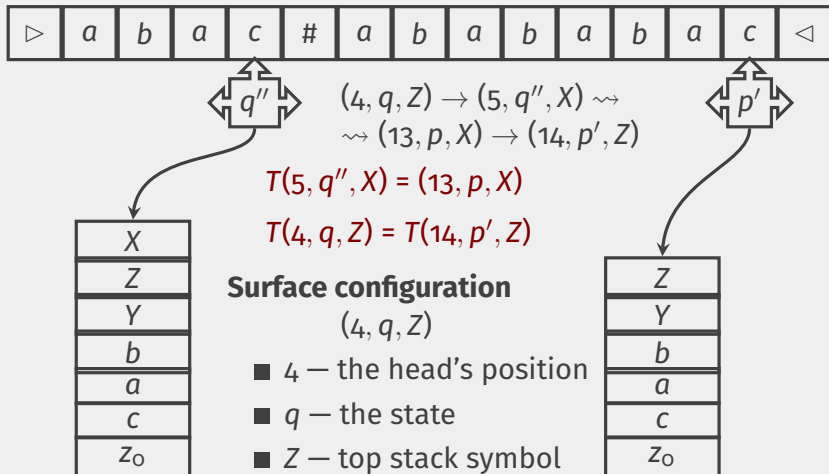
**Terminator** for  $(i, q, Z)$  is  $(j, p, Z)$  s.t.  $Z$  is popped.

# PROOF OF COOK'S THEOREM



**Terminator** for  $(i, q, Z)$  is  $(j, p, Z)$  s.t.  $Z$  is popped.

# PROOF OF COOK'S THEOREM



**Terminator** for  $(i, q, Z)$  is  $(j, p, Z)$  s.t.  $Z$  is popped.